

UNIVERSITÀ DEGLI STUDI ROMA TRE

Facoltà di Ingegneria
Corso di Laurea in Ingegneria Informatica

Tesi Di Laurea

Analisi e progettazione di strutture dati autenticate in memoria e in persistenza

Laureando

Matteo Giordano

Matricola 499253

Relatore

Prof. Maurizio Pizzonia

Anno Accademico 2017/2018

Ai miei genitori

Ringraziamenti

Ringrazio anzitutto il professor Maurizio Pizzonia, Relatore, Diego Pennino e Federico Griscioli: senza il loro supporto e la loro guida sapiente questa tesi non esisterebbe.

Un ringraziamento va anche ai miei genitori, per avermi sempre supportato per qualsiasi cosa avessi avuto desiderio di fare, contribuendo in larghissima parte a formare la persona che sono ora. Di questo sarò eternamente grato.

Vorrei inoltre fare uno speciale ringraziamento ai miei nonni, che hanno atteso eroicamente il raggiungimento di questo mio traguardo.

Introduzione

Negli ultimi anni, in funzione dell'esponenziale incremento di utenti che utilizzano dispositivi come computer, smartphone o simili, è cresciuta l'esigenza di servizi di *Cloud Computing* e, in particolare, di *Cloud Storage*. Quest'ultimo è un servizio di archiviazione e conservazione di dati sulla rete, dove appunto i dati stessi sono memorizzati su molteplici server. Fisicamente le risorse possono essere distribuite su più di un server in maniera, però, del tutto trasparente per l'utilizzatore finale. Il suo obiettivo ultimo tuttavia, risiede non solo nella volontà di memorizzare i dati sul cloud, ma soprattutto nella sicurezza informatica offerta da suddetti servizi. Sussiste infatti la sostanziale dicotomia tra i dispositivi utilizzati dagli utenti, sicuri, parte *fidata*, ma dalle capacità di storage limitate, e tutto l'ecosistema relativo al cloud, caratterizzato da storage illimitato ma *untrusted*, cioè non fidato. Un particolare aspetto, racchiuso all'interno di questo universo legato alla sicurezza informatica, in ambito di cloud storage, riguarda l'integrità dei dati. Un tale scenario può essere affrontato tramite l'utilizzo di *strutture dati autenticate*. Una ADS, infatti, rispecchiando la dicotomia di cui sopra, permette ad un client di verificare materiale non fidato: il *server*, con il compito di memorizzare i dati di interesse e di rispondere a richieste sugli stessi fornendone una *Prova* di validità, e il *client*, che opererà tali richieste al server, e vorrà poter verificare l'autenticità delle risposte.

Obiettivi

Gli obiettivi principali di progetto dunque hanno riguardato analisi, progettazione e realizzazione di una API, *Application Programming Interface*, riguardante l'implementazione efficiente di una struttura dati autenticata, basata su *Skip List*, cercando di

massimizzarne usabilità, prestazioni e sostenibilità e garantendo la compatibilità con un ambiente di utilizzo *concorrente*. Questa API è stata di supporto alla realizzazione di un protocollo per l'integrità di dati su cloud, nell'ambito di un progetto di ricerca svolto dal gruppo di sicurezza delle reti e dei sistemi informatici del Dipartimento di Ingegneria Informatica dell'Università degli Studi "Roma Tre". Di superiore importanza è, in secondo luogo, lo studio teorico comprensivo di attività di analisi e progettazione riguardante la possibilità di estendere le capacità dell'API, permettendo la memorizzazione e il mantenimento dell'ADS e dello storage in persistenza, prendendo ad esempio uno tra i più noti DBMS non relazionali, Cassandra.

Struttura della tesi

- **Analisi.** A seguito di opportune premesse riguardanti requisiti e casi d'uso e dimostrazioni sulle Skip List e sulla loro versione autenticata, è effettuato uno studio, dal punto di vista dell'analisi, della sezione riguardante strettamente l'API e successivamente della sezione relativa alla persistenza.

- **Progetto.** Conseguenza dei ragionamenti di analisi, sono qui affrontate tutte le scelte di progetto. Inizialmente si discute del design dell'API e dei loro dettagli di progettazione. Successivamente viene presentato il progetto relativo alla persistenza, con annessi studi teorici riguardo aspetti critici della progettazione stessa.

- **Realizzazione e testing.** Sono discusse le direttive di cui si è tenuto conto in fase di realizzazione e in fase di testing dell'intero lavoro di tirocinio.

Indice

Introduzione **iv**

Obiettivi iv

Struttura della tesi v

Indice **vi**

Elenco delle figure **viii**

1 Analisi **1**

1.1 Casi d'Uso 1

1.1.1 API 1

1.1.2 Persistenza 4

1.2 Modello di Dominio 4

1.3 Skip List 5

1.3.1 Definizione 6

1.3.2 Operazioni principali 8

1.4 Hashing Non-Commutativo 11

1.4.1 Funzioni 11

1.4.2 Resistenza a collisioni 12

1.5 Strutture dati autenticate 12

1.6 API 15

1.6.1 Skip List Autenticata 15

1.6.2 Proof 18

1.6.3 Query e Verifica 19

1.7 Persistenza 19

2 Progetto **21**

2.1 API 22

2.1.1 Interfaccia ADS 24

2.1.2 Interfaccia Proof 27

2.1.3 Nodo e schema hashing 29

2.1.4 Skip List Autenticata 31

2.1.5 Skip List Proof 36

2.2 Persistenza 41

2.2.1 Cassandra e NoSQL 41

2.2.2 Studi teorici di progettazione 42

2.2.3 Translator 46

2.2.4 CassandraConnector 47

2.2.5 ADSService 49

2.2.6 Comportamento asincrono 50

2.2.7 Caching 51

2.2.8 Bloom Filters 51

2.2.9 Modello con attori 53

2.2.10 Progettazione 54

3 Realizzazione e testing **58**

3.1 Realizzazione 58

3.2 Testing 58

Conclusioni e sviluppi futuri **60**

Problemi aperti e sviluppi futuri 60

Limiti concettuali su Proof 60

Ipotesi di soluzione con Modello a ruoli 61

Realizzazione progetto persistenza 62

Bibliografia **63**

Elenco delle figure

1.1 Il server riceve e accumula un certo numero di updates e ne richiede poi l'autenticazione tramite una Authentication Request. La risposta prevederà una Authentication Reply, non appena sarà avvenuta l'autenticazione . . . 2
1.2 Esempio di una Skip List. 7
1.3 **Ricerca dell'elemento 66** nella Skip List della figura 1.2. I nodi visitati e i collegamenti attraversati sono evidenziati con linee più spesse. Si noti che la ricerca, fallimentare, dell'elemento *70* produrrebbe lo stesso risultato. . . 9
1.4 **Caso d'uso di ADS**: l'ADS può essere caricato sul cloud e applicato su uno *storage* regolare, mentre il client memorizza localmente il Root Hash. Lo storage non dovrebbe essere a conoscenza dell'ADS. 13
1.5 **Query**: il client effettua una query e riceve oltre al risultato atteso, una Proof per quel risultato. Effettua dunque il calcolo localmente sulla Proof e ne verifica l'integrità grazie al Root Hash locale. 14
1.6 **Update**: il client richiede al server la Proof su un valore da aggiornare. Può, a questo punto, aggiornare il Root Hash locale e inviare il nuovo valore al server. Di quest'ultimo è, successivamente, il compito di aggiornare l'ADS, *refresh* in figura. 14
1.7 In figura è rappresentata un'astrazione di ciò che è presente sul server, preso isolatamente. La Skip List Autenticata, il cui elemento base è il Nodo, sta "sopra" lo storage classico. Genera opportune *Proof* in base alle richieste del client. 16
1.8 In alto a sinistra, lo *start node* ha il Root Hash, RH, dipendente dai valori m_n e dal loro ordinamento sulla Base List. 17

1.9 **Esempio di una proof**: i nodi costituenti la proof per la chiave 'key 3' sono evidenziati, così come i collegamenti necessari a ripercorrere gli *step* per ottenere il Root Hash. Si osservi che la Proof sarebbe stata la stessa per la chiave 'key 4' 18

2.1 Diagramma delle classi di progetto per la parte di autenticazione. Con un piccolo abuso di notazione sono stati presi in prestito alcuni elementi sintattici dal linguaggio *Kotlin*. 23
2.2 Singleton per funzioni di utilità per calcolo e restituzione di hash. 24
2.3 Diagramma delle classi di progetto per la parte di persistenza. 25
2.4 Esempio di un nodo. 30
2.5 Esempio di una pila. I nodi sono stati volutamente segnati con un identificativo, per facilitare la comprensione dell'immagine, ma non hanno alcun riferimento con la progettazione o la realizzazione reale dei nodi. (a) La pila contenente in ordine di attraversamento, dal basso verso l'alto, i passi compiuti per comporre il *path*. I blocchi della pila sono composti dal nodo di partenza di ogni passo, affiancato dalla stringa che indica la direzione del passo compiuto. (b) Sono evidenziati, nella Skip List, i nodi e le direzioni presenti nello stack, a seguito della ricerca dell'elemento 10. 32
2.6 Schema che mostra il funzionamento dei delta. Tramite primitive di aggiunta e cancellazione è popolata la pula della struttura con dei blocchi, ordinati cronologicamente. L'uso successivo di *applyDeltas()* applica in ordine questi delta e restituisce un'altra struttura, con stato di partenza equivalente al primo *snapshot*, ma con tutti i Δ_n applicati. La pila viene quindi svuotata, ed è dunque poi pronta ad accogliere nuovi delta. 35
2.7 Immagine che illustra un esempio di clonazione vista nella sua prima fase, rispetto alla Skip List nell'esempio. E' possibile poi da questo stato clonare tutti i nodi e provvedere alla sistemazione di tutti i collegamenti. 36
2.8 **(a)** Esempio di una Skip List Autenticata e **(b)** sua rispettiva possibile rappresentazione in persistenza su Cassandra. Ogni blocco, formato da una coppia chiave-valore di colonna, contiene le informazioni di un solo nodo. . 46
2.9 Diagramma di sequenza per l'operazione di check sulla presenza di una chiave. 54

2.10 Diagramma di sequenza per l'operazione di ricerca di una chiave. 55
2.11 Diagramma di sequenza per l'operazione di applyDeltas(). Lo schema mostra un particolare esempio in cui devono essere effettuate due operazioni, una di cancellazione e una di aggiunta. E' mostrata solo la parte di cancellazione in quanto la parte di aggiunta è analoga. La creazione di blocchi attuata dal Translator è implicita. 56
2.12 Diagramma di sequenza per l'operazione di calcolo del Root Hash. 56
2.13 Diagramma di sequenza per l'operazione di ottenimento di una Proof valida per una chiave. 57

Capitolo 1

Analisi

1.1 Casi d'Uso

1.1.1 API

La realizzazione dell'API, *Application Programming Interface*, è da contestualizzarsi all'interno dell'attività progettuale di tirocinio, svolta assieme al collega Gianmaria Del Monte. Il sistema preso qui in esame, è uno studio e realizzazione di protocollo efficiente per l'integrità dei dati su Cloud nell'ambito di un progetto di ricerca [FGP18] del professor Maurizio Pizzonia, svolto a più livelli di astrazione. Si richiama qui la parte svolta dal collega, di cui sul suo scritto [Mon18] è operata una trattazione più dettagliata, con lo scopo di definire casi d'uso e requisiti necessari.

Il contesto, in breve, prevede un numero variabile di *client*, con una disponibilità limitata in termini di memorizzazione, *storage*, ma che necessita di salvare una grande mole di dati. Questo sarà reso possibile da un *server*, considerata parte non fidata, il cui compito comprenderebbe anche l'attuazione del protocollo, delegando a particolari client, detti *autenticatori*, il compito di autenticare, appunto, le richieste provenienti dai client. Questi ultimi provvederanno all'autenticazione tramite firme, operando una criptazione asimmetrica, verificabili tramite dei *certificati*, noti a tutti i client. L'autenticazione di un valore relativo ad una richiesta di un client, tuttavia, dipenderebbe dall'avvenuta firma delle richieste precedenti. La forza del protocollo sta nella possibilità di attuazione di una architettura parallela di autenticatori, i quali sarebbero in grado

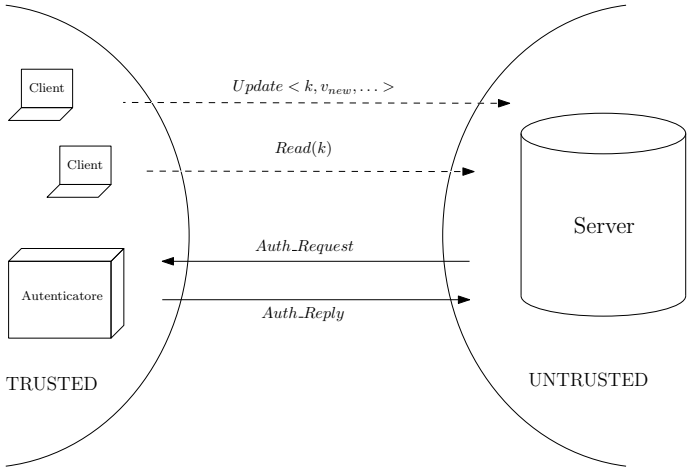


Figura 1.1: Il server riceve e accumula un certo numero di updates e ne richiede poi l'autenticazione tramite una Authentication Request. La risposta prevederà una Authentication Reply, non appena sarà avvenuta l'autenticazione

di autenticare un valore r_i, anche se il valore precedente r_{i-1} non è ancora stato firmato. Dunque intuitivamente, l'autenticazione può avvenire in maniera *condizionata* rispetto alla *catena* di valori precedenti, permettendo al server di richiedere una autenticazione quando la precedente non è ancora finita.

La gestione del lato server e dell'ecosistema comprendente gli autenticatori è stata gestita tramite l'utilizzo del Modello ad Attori. Si vuole qui solo contestualizzare l'ambito di utilizzo dell'API, con lo scopo di giustificarne i casi d'uso, pertanto per una trattazione approfondita si veda [Mon18]. All'interno del server, "vivranno" attori tra i quali *ADSManager*, con il compito di gestione del dataset, *ServerUpdate*, delegato alla gestione delle richieste di update da parte dei client, e *ServerRead*, per le richieste di read. All'interno dell'autenticatore, d'altro canto, saranno presenti attori tra i quali, *Authenticator*, con il ruolo di coordinazione generale degli altri attori presenti, *RootHashCalculator*, delegato al semplice calcolo del Root Hash di una Proof a seguito dell'applicazione sulla stessa di un blocco di update, e *ProofVerifier*, il quale verifica la validità del Root Hash[1] antecedente l'applicazione degli update.

[1]Concetti ancora poco chiari qui esposti saranno affrontati esaustivamente nei successivi paragrafi. Tuttavia è necessario farne menzione per giustificare i casi d'uso col giusto livello di dettaglio.

Fatte queste premesse, di seguito si elencano i casi d'uso della libreria:

- Una struttura dati autenticata, impostata come *key-value store*, in grado di fornire operazioni di lettura e di scrittura, mantenuta all'interno di ADSManager.
- Una forma di prova di validità, nel seguito *Proof*, per una o più chiavi presenti nella struttura, utilizzata da ProofVerifier e RootHashCalculator.
- Possibilità di ottenere dalla struttura dati autenticata il *digest*, nel seguito *Root Hash*, relativo a tutte le coppie chiave-valore presenti e al loro ordine di collocamento.
- Possibilità di ottenere dalla *Proof*, il *Root Hash* relativo alla chiave o all'insieme di chiavi sulle quali è costruita, oggetto di calcolo anche qui per ProofVerifier e RootHashCalculator.
- Capacità della *Proof* di aggiornarsi rispetto agli update che avvengono sulla struttura dati autenticata principale, per poter fornire il *Root Hash* aggiornato. Operazione necessaria a RootHashCalculator nell'iter del processamento di una Proof, lato autenticatore.
- Una struttura dati autenticata in grado di applicare su se stessa una serie di update, e in grado di restituire una *Proof* cumulativa per questi update, cosi come per qualsiasi insieme di chiavi desiderato.

Oltre a i requisiti funzionali di cui sopra, altre specifiche di cui si è tenuto conto riguardano aspetti di *usabilità*, nel fornire un'interfaccia di chiara e di facile interpretazione ai fini dell'utilizzo dell'API, di *prestazioni*, al fine di mirare al curve fitting con la complessità logaritmica, minimizzando la complessità spazio-temporale, e di *sostenibilità*, per garantire alta manutenibilità, verificabilità e configurabilità della API.

Infine è doveroso menzionare i vincoli di progetto:

- Compatibilità della libreria con utilizzo concorrente.
- Utilizzo della Skip List come struttura di base per realizzare l'ADS.
- Linguaggio di programmazione orientato agli oggetti JVM-Oriented.

1.1.2 Persistenza

Per quanto riguarda la seconda parte, questa è da interpretarsi come una estensione dell'API, atta a fornire la possibilità di memorizzare la struttura dati autenticata in persistenza, continuando ad ottemperare ai requisiti sopra esposti. Ovvero dovrà essere possibile continuare a richiedere tutti i servizi all'API, con la sola differenza che la struttura, cosi come lo storage su cui è applicata, non sarà in memoria, ma in persistenza. Uno tra i principi cardine di questo studio consiste nel garantire la trasparenza dell'API rispetto a questa estensione. Pertanto essa non avrà bisogno di essere modificata a seguito dell'introduzione delle modifiche per la persistenza. Ovviamente il servizio dovrà essere affidabile, sempre attivo e dovrà avere la capacità di tollerare guasti, errori e fallimenti. Oltre a questo, l'obiettivo principale, per cui nei successivi capitoli è presente un'approfondito studio teorico, è quello dell'efficienza, cercando di operare caricamenti e query in maniera parallela piuttosto che sequenziale.
Anche per questa sezione è presente un vincolo di progetto, che riguarda l'interoperabilità con un database management system di tipo non relazionale.

1.2 Modello di Dominio

Si procede, dunque, in questa sezione, dipendentemente dai casi d'uso e dai requisiti funzionali e non, sopra esposti, all'enumerazione dei concetti significativi nell'ambito di un modello di dominio. I concetti qui esposti, saranno poi più dettagliatamente spiegati nei successivi paragrafi, ma sono qui anticipati con lo scopo di fornire una rappresentazione visuale di entità concettuali rilevanti. Ebbene, sicuramente sarà necessaria un'entità che rappresenti la struttura dati autenticata, la quale dovrà essere in grado di offrire servizi di lettura e scrittura, rispettivamente di read e update. Essa inoltre deve essere in grado di fornire le *Proof* necessarie ai client o agli autenticatori per effettuare la verifica circa l'integrità dei dati. Allo stesso tempo la *Proof* deve saper fornire il *Root Hash* relativo alle chiavi sulle quali è definita, e deve saper applicare degli update sulle suddette chiavi. Inoltre, intuitivamente, sarà necessario che due *Proof* per due diversi insiemi di chiavi, possano unirsi, tramite un *merge* dando vita ad una *Proof* valida per entrambi gli insieme di chiavi di partenza.

1.3 Skip List

Prima di entrare nel dettaglio dell'analisi riguardante le strutture dati autenticate, si ritiene necessaria una preliminare analisi della struttura che sta alla base di questa particolare realizzazione: la *Skip List*. In letteratura frequente è la scelta di altri tipi di struttura dati, come ad esempio alberi binari di ricerca bilanciati: si pensi agli alberi rosso-neri, una tra le più utilizzate implementazioni.

La scelta di adozione di questa particolare struttura, sebbene risulti vincolo di progetto, rispecchia molte necessità richieste in ambito di utilizzo. Di seguito se ne presentano le principali, in confronto con il corrispettivo rappresentato dagli alberi rosso-neri.

La Skip List risulta essere più gestibile e adatta per quanto riguarda accessi e/o modifiche concorrenti. Infatti nel caso degli alberi rosso-neri, il problema sussiste dal momento in cui le operazioni di modifica comportano sempre la necessità di bilanciamento della struttura. Questa operazione può interessare anche gran parte della struttura stessa, risultando talvolta anche largamente bloccante per altre eventuali operazioni effettuate parallelamente. Altri utenti infatti, potrebbero voler accedere, in scrittura o in lettura, a nodi che devono ancora essere processati e/o ribilanciati. Le performance di un albero rosso-nero, che, come detto, è una delle principali implementazioni di alberi di binari di ricerca, risulterebbero degradare linearmente per ogni nuovo utente concorrente. Realizzazioni più rigide di questa struttura comportano infatti lo stato bloccante su tutti i nodi della struttura durante la fase di bilanciamento, e, nonostante esistano realizzazioni più rilassate, queste non hanno un impatto rilevante nel miglioramento delle performance [FH07]. Letture e scritture in una Skip List risultano invece molto più localizzate e solo pochi nodi, quelli direttamente interessati, richiedono lo stato bloccante.

La Skip List, dunque, soddisfa la contemporanea necessità di avere una struttura dati ordinata e che sia *thread-safe*, cioè adatta ad un contesto concorrente.

E' inoltre opportuno precisare da un lato il maggiore spazio richiesto dalle Skip List rispetto ad alberi bilanciati, in quanto necessitano, in media, del doppio dei puntatori per nodo, e dall'altro però la loro estrema facilità di implementazione.

1.3.1 Definizione

Dopo averne brevemente motivato l'adozione, si procede con una più precisa definizione che possa aiutare il lettore a comprendere appieno il razionale retrostante questa struttura. La Skip List è una struttura dati probabilistica che permette veloci ricerche all'interno di una sequenza ordinata di elementi. Essa mantiene un insieme ordinato di n elementi in tempo $\mathcal{O}(logn)$.

Prendendo ad esempio una lista semplice e non ordinata di elementi, si noti che l'operazione di ricerca di un elemento venga soddisfatta in tempo $\mathcal{O}(n)$. Lo stesso vale anche nel caso di liste ordinate. Questo tuttavia è vero nell'ipotesi in cui sia possibile accedere alla lista solamente tramite un puntatore verso l'inizio della stessa. Nell'ipotesi invece di accesso randomico a qualsiasi elemento, è possibile effettuare un'operazione di ricerca in tempo $\mathcal{O}(logn)$. Ebbene questa è l'idea retrostante la Skip List.
In termini pratici, tutto questo è reso possibile grazie ad una gerarchia di sottosequenze, organizzate su più livelli collegati tra loro. Lo sviluppo dei vari livelli è in altezza e prevede che la sottosequenza più in basso, detta *Base List* contenga tutti gli n elementi, mentre ogni livello superiore è composto da un sottoinsieme degli elementi del livello immediatamente sottostante. Le operazioni di ricerca iniziano dal livello con meno elementi, ovvero quello all'apice della struttura, e procedono per salti, in gergo 'skip', di nodo in nodo, fino a che non vengono trovati due elementi consecutivi, uno minore e uno maggiore o uguale all'elemento ricercato. A questo punto, grazie ai collegamenti inter-livello è possibile spostarsi sulla sottosequenza immediatamente inferiore e proseguire la ricerca. Questo procedimento è ripetuto fino a giungere alla sequenza completa di elementi, ovvero al livello più basso. Ogni livello inoltre, agli estremi laterali, possiede due sentinelle, ovvero due nodi speciali, marcati con $+\infty$ e $-\infty$. La presenza di questi ultimi su ogni livello ha come conseguenza la formazione di due torri, che racchiuderanno a destra e a sinistra la struttura intera. Si può dimostrare però che ciò non è sempre completamente necessario, ma si tratta più di rigorosità nella formulazione. Il nodo sentinella $-\infty$ del livello all'apice della struttura, è detto *Start Node* ed è il nodo di partenza per ogni operazione.

Il principio secondo cui gli elementi sono saltati nelle operazioni che scorrono sulle varie sequenze, ovvero indirettamente il principio secondo cui un elemento debba o me-

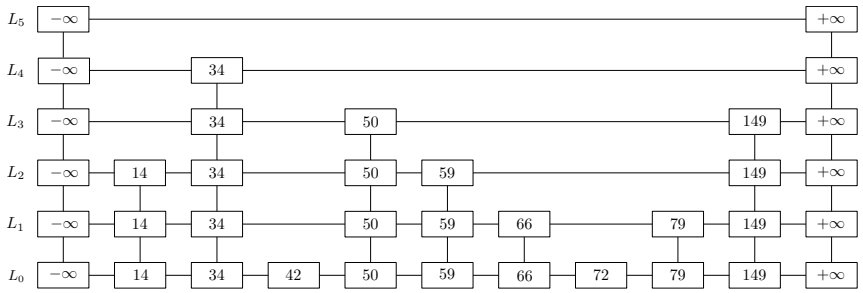


Figura 1.2: Esempio di una Skip List.

no essere presente anche nel livello immediatamente superiore, può essere randomico o deterministico. Il primo caso, di più frequente utilizzo, prevede una costruzione della struttura per strati. Il primo di questi, la Base List, contenente tutti gli n elementi, è una normale lista doppiamente o singolarmente concatenata. Dopodiché ogni elemento avrà una certa probabilità di essere presente anche nei livelli superiori. Normalmente la probabilità fissata a priori è pari a $1/2$, a tal punto che in letteratura si parli spesso di 'Fair coin flip', alludendo al lancio di una moneta regolare come evento determinante la presenza di un elemento in uno specifico livello. Pertanto l'aggiunta di un valore in una Skip List prevederebbe, intuitivamente, una sequenza di lanci di moneta per determinare in quante delle sottosequenze l'elemento debba essere presente, ossia indirettamente l'altezza della torre relativa a quello stesso elemento.

Prima di procedere con la descrizione dettagliata delle operazioni che fornisce, occorre introdurre ulteriore terminologia. Un nodo di un livello L_{i-1} contenente un elemento che non è presente al livello L_i è detto *plateau*. Un nodo che non presenta queste caratteristiche è detto *tower*. [GT01]

Per comodità delle future analisi si è deciso di adottare una terminologia simile a quella utilizzata da Goodrich e Tamassia nel loro scritto [GT01], a cui ci si rifarà anche nei successivi paragrafi: per ogni nodo v della sottosequenza L_i, è detto $elem(v)$ l'elemento memorizzato in v. Inoltre è detto $down(v)$ il nodo della sottosequenza L_{i-1} che memorizza lo stesso valore di v, a meno che il livello L_{i-1} non corrisponda alla Base List, poiché in questo caso $down(v) = null$. Inoltre è detto $right(v)$ il nodo della sottosequenza L_i immediatamente a destra di v. Per i nodi sentinella di tipo ∞ vale $right(v) = null$.

1.3.2 Operazioni principali

Dopo aver introdotto a grandi linee il razionale alla base della Skip List, si procede in questa sezione ad una più dettagliata enumerazione delle operazioni che supporta:

Ricerca Determina se un elemento è presente o meno nella Skip List.

Inserimento Inserisce un elemento.

Cancellazione Elimina un elemento.

1.3.2.1 Ricerca

L'operazione di ricerca di un elemento x in una Skip List inizia, come tutte le altre, dallo *Start Node*. La ricerca viene effettuata tramite l'alternanza di due azioni, *hop forward* e *drop down*, che rappresentano rispettivamente il movimento verso destra o verso il basso all'interno della struttura. Esse sono ripetute, sotto determinate condizioni, fino a che la ricerca non sia terminata, indipendentemente dall'esito, che potrà essere positivo o negativo.

La prima azione menzionata prevede movimenti verso destra fino a che non si raggiunge un nodo il cui valore sia il più grande minore o uguale a x. La seconda azione invece consiste semplicemente in uno spostamento verticale, al nodo immediatamente sottostante, a condizione che questo esista. In pseudocodice, i passi rilevanti delle due azioni, rispettivamente:

- **Hop forward**:

```
while elem(right(v)) < x do
    v ← right(v);
end
```

- **Drop down**:

```
if down(v) != null then
    v ← down(v);
else
    end search;
end
```

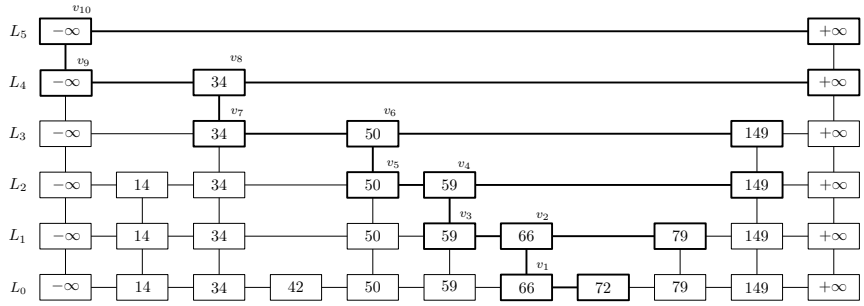


Figura 1.3: **Ricerca dell'elemento 66** nella Skip List della figura 1.2. I nodi visitati e i collegamenti attraversati sono evidenziati con linee più spesse. Si noti che la ricerca, fallimentare, dell'elemento *70* produrrebbe lo stesso risultato.

La ricerca consiste dunque in una serie di *hop forward* e *drop down*, fino a che non sia più possibile effettuare ulteriori *drop down*. Con questa sequenza di passi si raggiungerà un nodo sulla Base List: se vale che $elem(v) = x$ allora la Skip List contiene l'elemento x. Se questa condizione non risulta vera, allora il nodo raggiunto tramite questa sequenza di passi sarà il nodo sulla Base List con il valore più grande minore di x. Per costruzione, di conseguenza, $right(v)$ sarà il nodo sulla Base List il cui elemento è il più piccolo maggiore di x.

Il costo computazionale dell'operazione di ricerca è dato dalla somma dei costi di attraversamento di tutti i livelli. Tuttavia ogni livello, tranne il primo, subisce un attraversamento parziale, in quanto il nodo di partenza non è il primo nodo della sequenza.

Considerando che il numero di livelli in una Skip List con n elementi è, con alta probabilità[2], pari a $\mathcal{O}(logn)$, è dimostrabile che il costo di ricerca è, con alta probabilità, pari a $\mathcal{O}(logn)$.

1.3.2.2 Inserimento

L'operazione di inserimento è sempre preceduta da una operazione di ricerca. Dunque per inserire un elemento x, per prima cosa è effettuata una ricerca su x, con l'unica differenza che ogni volta che viene effettuato un *hop forward* o un *drop down* rispetto al nodo corrente v, viene aggiunto un elemento in una pila che registri il nodo corrente v e

[2]Un evento occorre con alta probabilità quando la sua probabilità dipende da un certo valore n ed essa tende a 1 quando n tende a ∞

l'azione associata. Dal momento in cui l'elemento x da inserire non è ancora presente, l'operazione di ricerca restituirà il nodo sulla Base List che dovrà essere immediatamente precedente al nodo da inserire. A questo punto sarà possibile inserire il nodo memorizzando il valore x. Dopodiché, nel caso di Skip List randomiche, viene valutata l'altezza della torre, ovvero in quali livelli superiori dovrà essere presente l'elemento, tramite una sequenza di tiri di moneta simulati. Per ogni lancio di moneta, nel caso in cui il risultato sia testa, viene utilizzato la pila per ripercorrere il percorso a ritroso e ottenere la posizione, del livello superiore, in cui dovrà essere inserito il nuovo nodo, tramite opportune operazioni di *pop*. Il procedimento continua fino a che il risultato non sia croce. Nel caso in cui la torre dovesse superare l'altezza corrente della struttura, si aggiunge un nuovo livello alla struttura, comprensivo, ovviamente, dei nuovi nodi sentinella ai suoi estremi. L'operazione di cancellazione ha, con alta probabilità, costo computazionale pari a $\mathcal{O}(logn)$.

1.3.2.3 Cancellazione

L'operazione di cancellazione di un elemento x prevede la ricerca di un elemento che sia minore di x ma contemporaneamente maggiore di ogni altro elemento nella Base List tra quelli minori di x. Come per l'operazione di inserimento, viene utilizzata una pila di supporto per tenere traccia dei passi effettuati dalla ricerca. Chiaramente, l'elemento immediatamente successivo al nodo restituito dall'operazione di ricerca, dovrà essere x, poiché in caso contrario l'elemento da cancellare non sarebbe presente nella Skip List, scenario di non successo, da segnalare con errore opportunamente. Dunque viene rimosso il nodo interessato, e tutti i nodi sulla stessa torre, ottenendone la posizione grazie ad opportune operazioni di *pop* dalla pila precedentemente riempita. Nel caso in cui la rimozione dovesse comportare un abbassamento globale della struttura, devono essere eliminati i nodi sentinella interessati. L'operazione di cancellazione ha, con alta probabilità, costo computazionale pari a $\mathcal{O}(logn)$.

1.4 Hashing Non-Commutativo

Si ritiene necessario, a questo punto, introdurre una speciale classe di funzioni: le funzioni crittografiche di hash. Si tratta di algoritmi matematici che mappano, cioè fanno corrispondere, dati di lunghezza arbitraria, comunemente chiamati *messaggi*, in un stringa binaria di dimensione fissa, detta *valore di hash*. L'output prodotto è solitamente chiamato *digest*. L'hash di un messaggio m è denotato $h(m)$ e varranno le seguenti proprietà: [Piz17]

- Per ogni m il calcolo di $h(m)$ è efficiente;
- Dato un H, è computazionalmente difficile trovare m tale che $H = h(m)$;
- Dato h, è computazionalmente difficile trovare un $m \neq m^{'}$ tale che $h(m) = h(m^{'})$;
- Dato h e m, è computazionalmente difficile trovare un $m \neq m^{'}$, tale che $h(m) = h(m^{'})$.

E' altrettanto vero che la funzione può essere applicata a due o più messaggi. Pertanto data una coppia di valori a e b, sarà $H = h(a, b)$ il digest prodotto dalla funzione di hash. Lo stesso è applicabile anche su triple, e più in generale su n-ple di lunghezza qualsiasi. Per effetture il digest di una sequenza $(x_1, x_2, ..., x_m)$, dovrà essere effettuato il seguente calcolo: $h(x_1, h(x_2, ...h(x_{m-2}, h(x_{m-1}, x_m))...))$. [GT01]

1.4.1 Funzioni

Per lo scopo, possono essere adottati vari tipi di funzione di hash, sui singoli valori. Si è ritenuto importante garantire un certo livello di flessibilità, permettendo una scelta all'interno dell'API. Le funzioni tra le quali è possibile scegliere appartengono ad una famiglia delle funzioni crittografiche di hash, le *Secure Hash Algorithms*, abbreviato *SHA*. In particolare è possibile scegliere tra lo SHA-1, funzione di 160 bit la cui vulnerabilità crittografica è nota, lo SHA-256 e lo SHA-512, resistenti a pericolosi attacchi come il *Length extension attack*, le quali differiscono nella dimensione dell'output.

1.4.2 Resistenza a collisioni

L'utilità di una funzione crittografica di hash all'interno di una struttura dati autenticata, di cui si parlerà più approfonditamente nel successivo paragrafo, è data dalla resistenza alle collisioni. Ovvero data una coppia di valori (a, b), risulta computazionalmente difficile trovare una coppia $(c, d) \neq (a, b)$ tale per cui $h(a, b) = h(c, d)$. Infatti il costo computazionale richiesto per trovare una coppia di collisione (c, d) sarebbe troppo elevato anche per un supercomuter.
La specifica di non commutatività della funzione di hash, serve a sottolineare l'esclusione della proprietà di commutatività che potrebbe essere inclusa per semplificare i processi di verifica effettuati dall'utente. In questo studio tuttavia essa non è permessa, causa vincoli di progetto, e dunque formalmente $h(a, b) \neq h(b, a)$ per ogni a e b.

1.5 Strutture dati autenticate

Una struttura dati autenticata, abbreviato *ADS*, è una struttura dati che è adatta a controlli sulla sua integrità, cosi come su porzioni di essa. Memorizza valori e associa una funzione crittografica di hash h con il suo contenuto. L'output prodotto da questa funzione è detto *Root Hash* e possiede lunghezza finita. Tramite una ADS, dunque, un client può rilevare piccoli cambiamenti all'interno di un *dataset* anche molto grande, in maniera efficiente. La sua applicazione in ambiente cloud risulta particolarmente proficua in quanto generalmente permette di ottenere elevate garanzie su sicurezza, autenticazione e verifica di integrità dei dati. Tutto questo rispettando la dicotomia tra l'ambiente di storage locale, limitato ma fidato, *trusted*, e l'ambiente cloud, virtualmente illimitato, ma non fidato. Il razionale alla base consiste nel memorizzare un grande *dataset* sul cloud, equipaggiandolo con una ADS, e mantenere soltanto il root hash localmente.

Un'ADS, dunque, è una struttura dati dove una serie di operazioni può essere eseguita da un entità non fidata, ma dove i risultati possono essere sottoposti a controlli sull'autenticità. Questo è possibile grazie al fatto che l'entità che esegue le operazioni, oltre a restituire il risultato delle stesse, produrra una prova di validità, *Proof*, che sarà utilizzabile per verificare ogni singolo risultato.

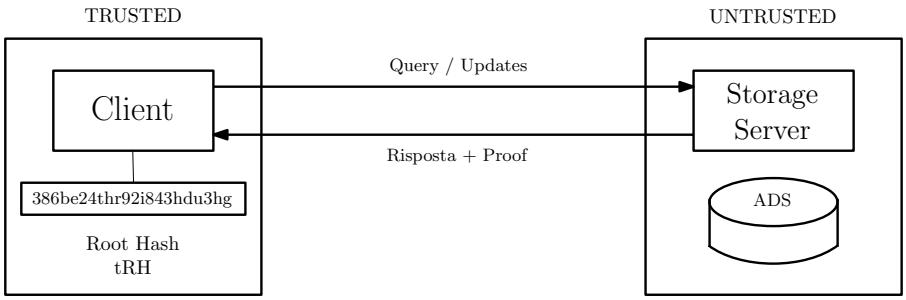


Figura 1.4: **Caso d'uso di ADS**: l'ADS può essere caricato sul cloud e applicato su uno *storage* regolare, mentre il client memorizza localmente il Root Hash. Lo storage non dovrebbe essere a conoscenza dell'ADS.

Una ADS, come già accennato in precedenza, può comportare la presenza di almeno due ruoli: il *server*, con il compito di memorizzare i dati di interesse e di rispondere a richieste sugli stessi fornendone una *Prova* di validità, e il *client*, che opererà tali richieste al server, e vorrà poter verificare l'autenticità delle risposte. In particolare, ad ogni istante temporale, il client memorizzerà localmente un piccolo *digest*, il quale funge da sorta di riassunto dell'intero contenuto dell'ADS in quel determinato istante temporale. Quando il client effettua una query, il server calcola la risposta e la restituisce assieme alla Proof. Il check sulla Proof effettuato da parte del client, tuttavia, può essere effettuato proprio grazie al digest che memorizza localmente.

- Le varie query, effettuate dal client, restituiranno oltre ai valori richiesti, anche una prova, detta *Proof*. Tramite la Proof ricevuta e il Root Hash locale, il client potrà verificare l'integrità del risultato della query.

- Gli updates tuttavia, andranno a cambiare il *dataset* remoto, e dunque l'ADS presente sul cloud. Il client inoltre aggiorna il suo Root Hash locale con quello nuovo prodotto dall'ADS.

Dunque, update effettuati da un qualsiasi client, su un qualsiasi valore, si ripercuoteranno sul Root Hash dell'intera struttura, permettendo il rilevamento di queste modifiche ad ogni altro client, in tutte le successive query.

Le più diffuse implementazioni di strutture dati autenticate sono:

- Merkle Hash Tree (MHT).

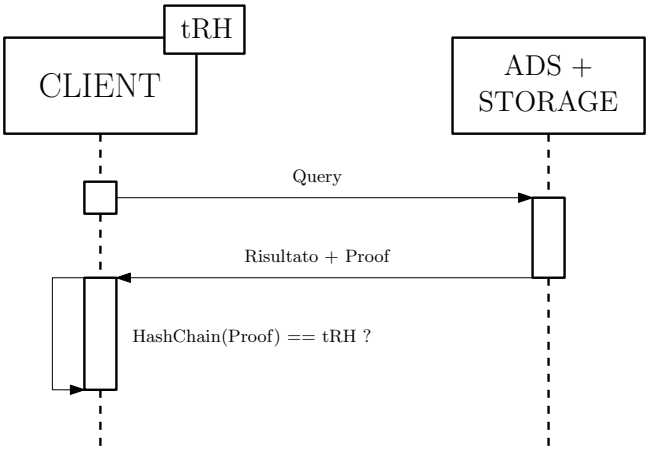


Figura 1.5: **Query**: il client effettua una query e riceve oltre al risultato atteso, una Proof per quel risultato. Effettua dunque il calcolo localmente sulla Proof e ne verifica l'integrità grazie al Root Hash locale.

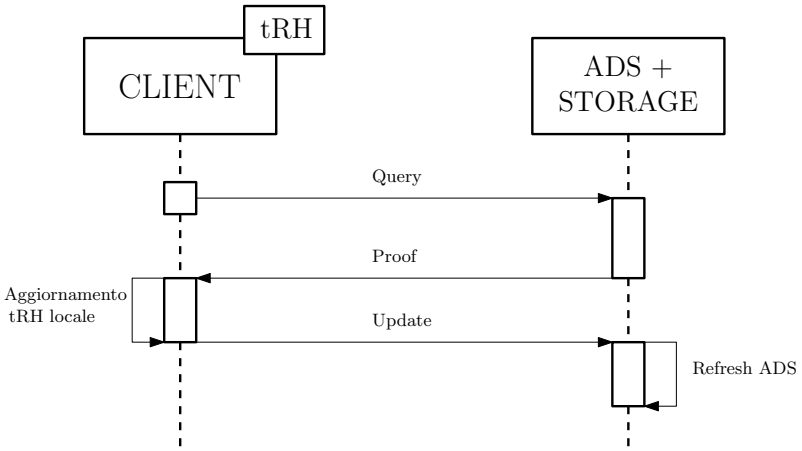


Figura 1.6: **Update**: il client richiede al server la Proof su un valore da aggiornare. Può, a questo punto, aggiornare il Root Hash locale e inviare il nuovo valore al server. Di quest'ultimo è, successivamente, il compito di aggiornare l'ADS, *refresh* in figura.

- Skip list autenticate.

Il funzionamento di quest'ultima sarà approfondito nei successivi paragrafi.

1.6 API

Riallacciandosi a quanto detto nella sezione riguardante il modello di dominio, è possibile a questo punto aggiungere informazioni più precise. L'API girerà sul server, e dovrà consentire la gestione della simbiosi tra lo storage e, appunto, l'ADS. L'entità rappresentante la struttura dati autenticata, in base a vincoli di progetto, dovrà basarsi sulla Skip List. Si tratterà, dunque, di un *key-value store* capace di offrire servizi di update e read, generando anche le opportune *Proof* in base al particolare tipo di richiesta del client. L'elemento costituente della Skip List Autenticata, sarà il nodo, in grado sia di memorizzare la chiave relativa alla torre in cui si trova, sia gli hash funzione dei nodi adiacenti. In questo modo la struttura potrà comportarsi come un'ADS a tutti gli effetti, in ottemperanza con i requisiti e con i casi d'uso. Il funzionamento preciso dell'API sarà affrontato con il dovuto dettaglio nella sezione di Progetto, mentre qui si vuole soprattutto spiegare il funzionamento concettuale e la collaborazione tra le parti.

1.6.1 Skip List Autenticata

Una Skip List Autentiticata, come deducibile dalla figura 1.7, è una particolare implementazione di un ADS, che si basa su Skip List, e sulla quale è applicata una funzione crittografica di hash h. La struttura dunque aggiungerà alla funzione di memorizzare un insieme di valori, in maniera ordinata, le caratteristiche tipiche di un ADS. Per permettere dunque sia l'ordinamento dei dati, sia l'applicazione di h, si prende in esame una struttura basata su coppie chiave-valore. In questo modo le chiavi permetteranno l'ordinamento dei dati, e i valori permetteranno l'applicazione dello schema di hashing, approfondito nel prossimo paragrafo. Ogni nodo conterrà, oltre alla chiave relativa alla torre su cui giace, un hash dei dati di alcuni altri nodi adiacenti. Intuitivamente, il suo hash potrebbe essere in funzione di $w = right(v)$ e $u = down(u)$. Questo schema confluirà nel Root Hash, ovvero nel *digest* dell'intera Skip List, memorizzato nello Start

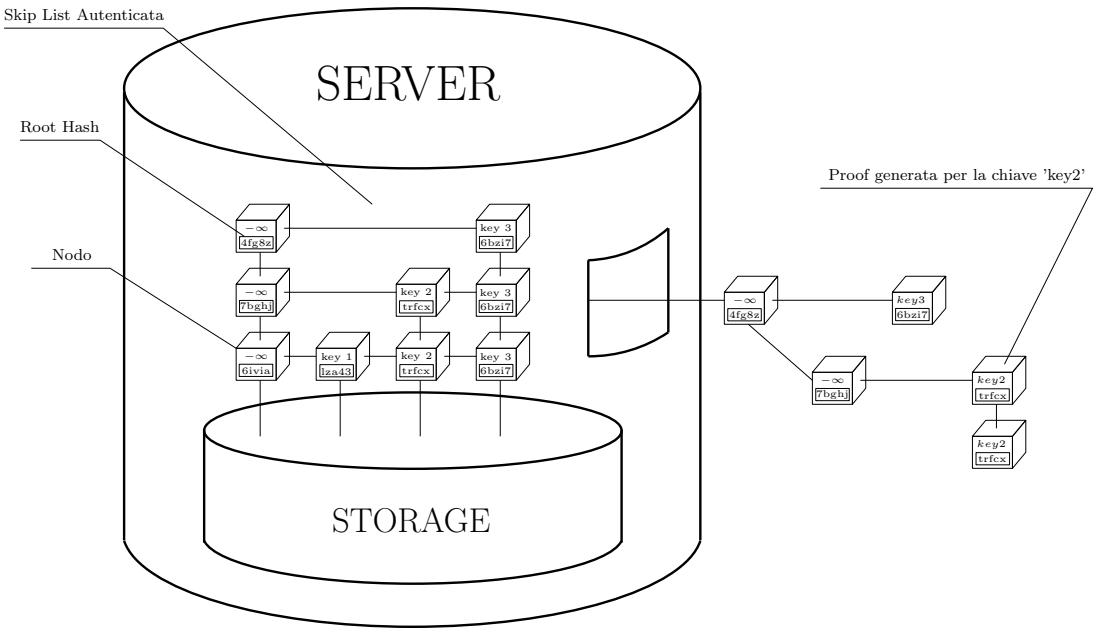


Figura 1.7: In figura è rappresentata un'astrazione di ciò che è presente sul server, preso isolatamente. La Skip List Autenticata, il cui elemento base è il Nodo, sta "sopra" lo storage classico. Genera opportune *Proof* in base alle richieste del client.

Node. Il Root Hash, per definizione, sarà rigidamente dipendente sia dai valori di ogni nodo che dalla loro posizione reciproca. Da questo calcolo sono esclusi i nodi sentinella.

1.6.1.1 Schema hashing

Il calcolo del Root Hash dunque è strettamente legato allo schema di hashing scelto, ovvero alla modalità secondo cui i singoli nodi calcolano il loro hash locale in funzione dei nodi adiacenti. Per questa analisi, si è deciso di seguire lo schema suggerito da Tamassia [GT01], con alcune piccole variazioni. Sia $f(v)$ la funzione di hash di ogni singolo nodo, allora varrà che:

1. Se $u = null$, e cioè se il nodo v si trova sulla Base List:
   - (a) Se w è un nodo *tower*, allora $f(v) = h(elem(v))$.
   - (b) Se w è un nodo *plateau*, allora $f(v) = h(h(elem(v)), f(w))$.
2. Se $u \neq null$, e cioè se il nodo v non si trova sulla Base List:

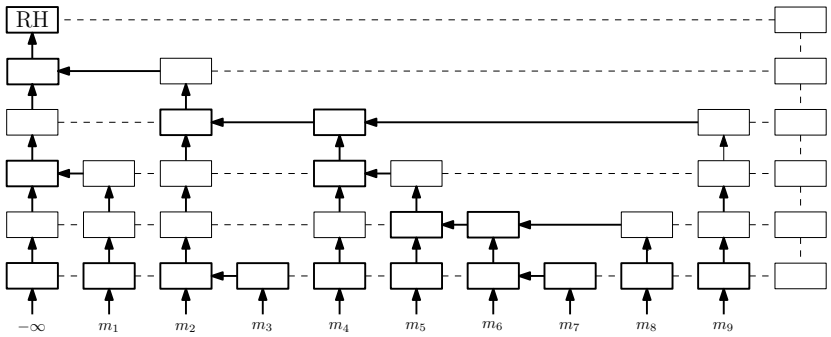


Figura 1.8: In alto a sinistra, lo *start node* ha il Root Hash, RH, dipendente dai valori m_n e dal loro ordinamento sulla Base List.

- (a) Se w è un nodo *tower*, allora $f(v) = f(u)$.
- (b) Se w è un nodo *plateau*, allora $f(v) = h(f(u), f(w))$.

1.6.1.2 Operazioni

In quanto ADS, la Skip List Autenticata offrirà due tipologie di operazione:

Update Offre la possibilità di fare inserimenti o cancellazioni.

Read Operazioni di lettura che permettono di ottenere un valore assieme alla sua Proof, grazie alla quale sarà possibile verificare l'integrità del risultato.

1.6.1.3 Update

In base a quanto osservabile dai casi d'uso, l'*update* è l'operazione di primaria importanza. Nel momento in cui si vuole modificare, aggiungere o cancellare un valore nello *storage*, la Skip List Autenticata deve aggiornarsi internamente, proprio a causa di questa modifica. Questo è possibile tramite l'operazione di update. Il client dunque, interagento con la struttura, dovrà specificare il tipo di operazione da svolgere (modifica, aggiunta o cancellazione) e i parametri necessari per eseguirla. Nei primi due casi si tratterà di una coppia chiave-valore (k, v), nel caso di cancellazione basterà specificare la chiave k. A seguito dell'invocazione di questa operazione sull'ADS, i valori di hash dei singoli nodi devono essere aggiornati per riflettere il cambiamento

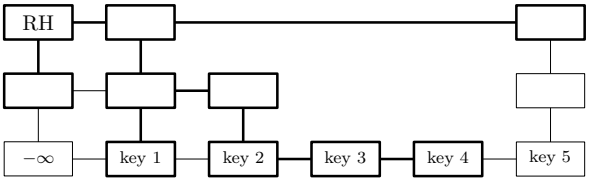


Figura 1.9: **Esempio di una proof**: i nodi costituenti la proof per la chiave 'key 3' sono evidenziati, così come i collegamenti necessari a ripercorrere gli *step* per ottenere il Root Hash. Si osservi che la Proof sarebbe stata la stessa per la chiave 'key 4' .

che è avvenuto, e con essi il Root Hash. Nel caso di inserimento o cancellazione, la prima parte di esecuzione dell'operazione è identica a quella di una Skip List semplice, con l'ovvia precisazione che i movimenti avverranno non più sui valori, ma sulle chiavi. Successivamente all'aggiunta/cancellazione del nodo interessato, viene utilizzata la pila precedentemente costituita per aggiornare gli hash dei nodi. Saranno infatti solamente i nodi presenti nella pila, ed eventualmente i suoi vicini di destra, grazie allo schema di hashing precedentemente presentato, a dover essere aggiornati. Pertanto ripercorrendo a ritroso tramite operazioni di *pop* la pila, la struttura dovrà ricalcolare le varie $f(v)$ sia dei nuovi nodi sia di quelli modificati. L'aggiunta in termini di costo computazionale dovuta a questa ulteriore operazione di aggiornamento degli hash, è proporzionale al numero di elementi presenti nella pila. Con alta probabilità l'operazione sarà eseguita in $\mathcal{O}(logn)$.

1.6.2 Proof

Prima di analizzare l'operazione di query, è necessario soffermarsi sul concetto di *Proof*, sopra solo accennato. La Proof non è altro che un insieme di *step* da percorrere, secondo un preciso criterio che rispecchia lo schema di hashing presente nella struttura dati autenticata, per ottenere il Root Hash. Essa è utilizzato per ottenere una prova, appunto, della validità di un particolare elemento restituito da una query. Infatti oltre a ricevere un generico valore x, corrispondente ad una chiave k, il client ottiene anche una *Proof(k)*, e potrà dunque verificare localmente la corrispondenza tra il proprio Root Hash locale, *trusted Root Hash*, e quello ottenibile seguendo l'opportuna serie di passi sulla Proof ottenuta, *untrusted Root Hash*.

1.6.3 Query e Verifica

Analizzato il concetto di Proof è possibile ora inserirlo nel più ampio contesto di una query. Un client, a seguito dell'esecuzione di una query di tipo read su una determinata coppia chiave-valore (k, v), otterrà una *Proof(k)* relativa a quella chiave. Il client, che memorizza localmente il *root hash* di struttura, ripercorrerà gli *step* opportuni sulla proof, grazie alla conoscenza della particolare funzione crittografica di hash h correntemente attiva sull'ADS. (Deve cioè effettuare operazioni di hash, nel dato ordine, di tutta una sequenza di hash.) Otterrà dunque il Root Hash presente sul cloud e potrà verificare che questo corrisponda a quello locale. Nel caso in cui questo non accada, vorrà dire che l'integrità del valore ottenuto non è garantita, in quando sono state effettuate ulteriori modifiche sulla struttura, ovvero sul *dataset*, non desiderate.

1.7 Persistenza

In questa sezione vengono sottolineati alcuni dettagli di carattere generale, per approfondire concetti espressi nella precedente sezione di analisi di casi d'uso e requisiti. Si è detto precedentemente che la memorizzazione in persistenza della Skip List Autenticata deve innestarsi perfettamente all'interno dell'API, cioè deve estenderne le capacità. Il funzionamento delle stesse pertanto dovrà rimanere invariato con l'unica differenza che la struttura dati, piuttosto di essere in memoria, sarà in persistenza. Dunque, prima dell'esecuzione di ogni operazione, sarà necessario interagire con il database NoSQL per ottenere le informazioni necessarie all'esecuzione dell'operazione stessa. La logica di interazione con il database e il funzionamento della politica di restituzione delle informazioni sono, in questa fase di analisi, incapsulate a scatola nera all'interno di un *Sistema*, rendendo per ora ignoto il funzionamento interno, col fine di concentrarsi su aspetti concettuali preliminari.
In primo luogo sarebbe auspicabile che, dovendo essere quella della persistenza una estensione dell'API, il *Sistema* debba gestire il minimo delle informazioni possibile. Ossia dovrebbe provvedere alla restituzione delle informazioni necessarie e sufficienti, all'esecuzione di una determinata operazione, evitando di dover gestire materiale inutile. In merito a questo, è da sottolineare che la struttura dati autenticata è stata pensata

per necessitare, sia nelle operazioni di retrieve, sia in quelle di update, soltanto dei nodi diretti interessati. Approfondimenti sull'argomento sono affrontati nella sezione di progettazione relativa alla persistenza. Collegato a ciò è il discorso sulla trasparenza dell'introduzione della persistenza rispetto alla libreria in-memory. In base a quanto detto, le varie operazioni necessitano solo di una parte della struttura, ovvero di un sottoinsieme di nodi. Fornendo, preliminarmente all'esecuzione dell'operazione, questi nodi, strutturati opportunamente, l'API potrà continuare ad operare esattamente come se la struttura fosse contenuta interamente in memoria. La complessità della gestione dell'ADS sarà tutta addossata al *Sistema*. Inoltre, per quanto riguarda l'efficienza, il punto principale da rispettare riguarda la possibilità, per il *Sistema*, di poter effettuare le query per ottenere le informazioni necessarie in maniera parallela piuttosto che sequenziale. Una query dovrà dunque essere indipendente dal risultato della query precedente, eseguibile cioè in maniera parallela. Nel caso di query sequenziali infatti, la complessità computazionale di qualsiasi blocco di operazione di restituzione sarebbe direttamente proporzionale al numero di elementi restituiti cioè ordine di $\mathcal{O}(n)$. Nel caso parallelo invece, indipendentemente da quanti elementi saranno restituiti, l'operazione potrà essere eseguita in un solo round.

Fatte dunque queste premesse preliminari, nella fase di progettazione verranno illustrate e motivate le scelte progettuali interne al *Sistema*, oltre all'aggiunta di ulteriori specifiche e osservazioni, rilevabili solo però in questa fase.

Capitolo 2

Progetto

A seguito della fase di analisi appena condotta, si passa all'ambito più strettamente progettuale. Scopo di questa sezione introduttiva è quello di riallacciarsi ai requisiti già esposti in fase di analisi, aggiungendo obiettivi specifici di progettazione. Per quanto riguarda l'API, si è già parlato dei requisiti funzionali, cosi come si è sottolineata la necessità di utilizzo in *concorrenza*.
Si pone ora l'accento invece su alcuni obiettivi progettuali di cui si è tenuto conto:

1. **Prestazioni**. Si vogliono garantire complessità computazionali compatibili e coerenti con quelle illustrate in fase di analisi per la Skip List, minimizzando lo spazio utilizzato dalla struttura dati, cosi come garantire ottimi tempi di risposta e throughput.

2. **Usabilità**. Si vuole una interfaccia ben definita, chiara, di facile interpretazione e stabile, al fine di ottimizzarne l'utilizzo.

3. **Sostenibilità**. Progettazione e realizzazione devono permettere alti livelli di manutenibilità per poter operare ottimizzazioni, miglioramenti e riparazioni. Di pari importanza sono la verificabilità e la configurabilità dell'API.

Per quanto riguarda la sezione sulla persistenza, invece, si ricordano i due requisiti fondamentali. Il primo riguarda la trattazione della stessa come un'estensione dell'API, il secondo invece sottolinea la caratteristica di trasparenza dell'introduzione di questa

estensione, rispetto alla libreria stessa. Si pone anche qui l'accento sugli obiettivi progettuali principali di cui si è tenuto conto:

1. **Efficienza**. L'obiettivo principale riguarda l'impostazione della restituzione e caricamento di elementi da e sul database in maniera parallela piuttosto che sequenziale. Inoltre è auspicabile un sistema che sappia lavorare sul minimo di informazioni necessarie all'esecuzione delle operazioni, evitando il più possibile traffico di elementi inutili. L'architettura complessiva, infine, dovrebbe poter lavorare con un sistema di caching efficiente, per ridurre il traffico di informazioni generale e dunque il carico gravante sulle query con il database.

2. **Affidabilità**. Si vuole un sistema disponibile, sempre attivo e che fornisca la capacità di essere ripristinato a seguito di guasti, errori o fallimenti.

Seguono i diagrammi della classi dell'intera progettazione per fornire un contesto alla successiva analisi dettagliata delle singole parti[1].

2.1 API

Come già deducibile dal Class Diagram in figura 2.1, l'API consiste fondamentalmente di due interfacce, *stabili*, *comuni* e *unificate*, che forniscono *Protected Variation* rispetto a rivisitazioni e ottimizzazioni. Pertanto è stato fornito all'utilizzatore un punto di contatto singolo, una *Facade*, con le librerie. Le implementazioni di queste due interfaccie sono soggette a due vincoli di progetto riguardanti il tipo da utilizzare per le chiavi, ovvero una stringa di caratteri, e per gli hash, ovvero array di byte e/o stringa di caratteri. Esse di basano entrambe sul *Node* come elemento costituente di base. Per le funzioni di hash, esse si affidano ad un Singleton contenente funzioni di utilità, *HashUtils*. Prima di passare ad un'analisi dettagliata delle varie parti, si sottolinea la dipendenza della Skip List Autenticata dall'interfaccia della Proof, e *non* dalla sua implementazione. Questo è stato preferito per ridurre l'accoppiamento generale, fornendo uno strato di *Indirection*, in quanto la Skip List Proof, può subire cambiamenti

[1] Per tutti i diagrammi delle classi mostrati, la molteplicità delle varie associazioni è sottintesa essere pari a 1.

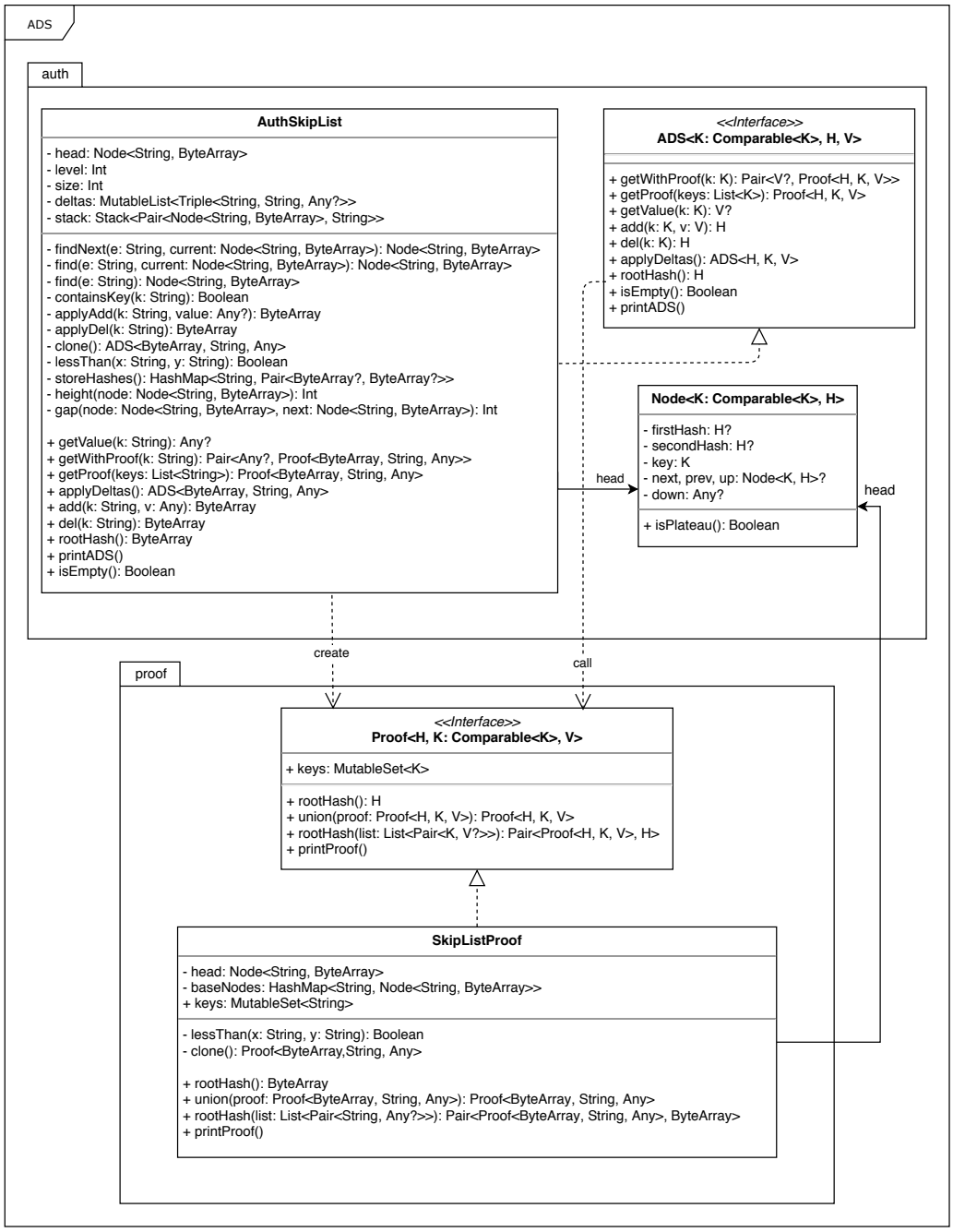


Figura 2.1: Diagramma delle classi di progetto per la parte di autenticazione. Con un piccolo abuso di notazione sono stati presi in prestito alcuni elementi sintattici dal linguaggio *Kotlin*.

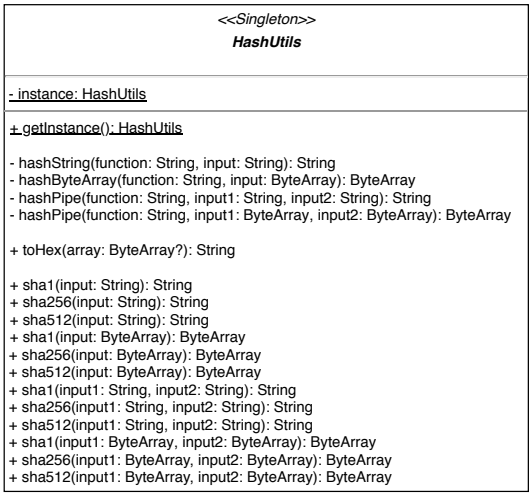


Figura 2.2: Singleton per funzioni di utilità per calcolo e restituzione di hash.

e ottimizzazioni interne, senza ripercuotersi direttamente sulla Skip List Autenticata. Nei successivi paragrafi si spiegherà più nel dettaglio ogni parte attiva all'interno del progetto di questa API, illustrandone e motivandone tutte le scelte progettuali rilevanti.

2.1.1 Interfaccia ADS

L'interfaccia dell'ADS, è stata resa parametrica rispetto a tre valori:

- **H**: il tipo dei vari *hash* presenti all'interno dei nodi della struttura.
- **K**: il tipo delle *chiavi* dei nodi.
- **V**: il tipo dei valori presenti nello *storage*.

Questa scelta rende l'interfaccia applicabile parametricamente rispetto alla scelta dei suddetti tipi, caratteristica utile per garantire flessibilità.

Le operazioni fornite dall'ADS, sulla base di requisiti e casi d'uso già discussi in fase di analisi, sono[2]:

getWithProof(k: K): Pair<V?, Proof<H, K, V>> Restituisce il valore v associato alla chiave k, assieme alla *Proof* per quella chiave. Questa operazione è

[2]Il punto interrogativo accanto ad un parametro sta ad indicare che quel parametro può essere anche nullo

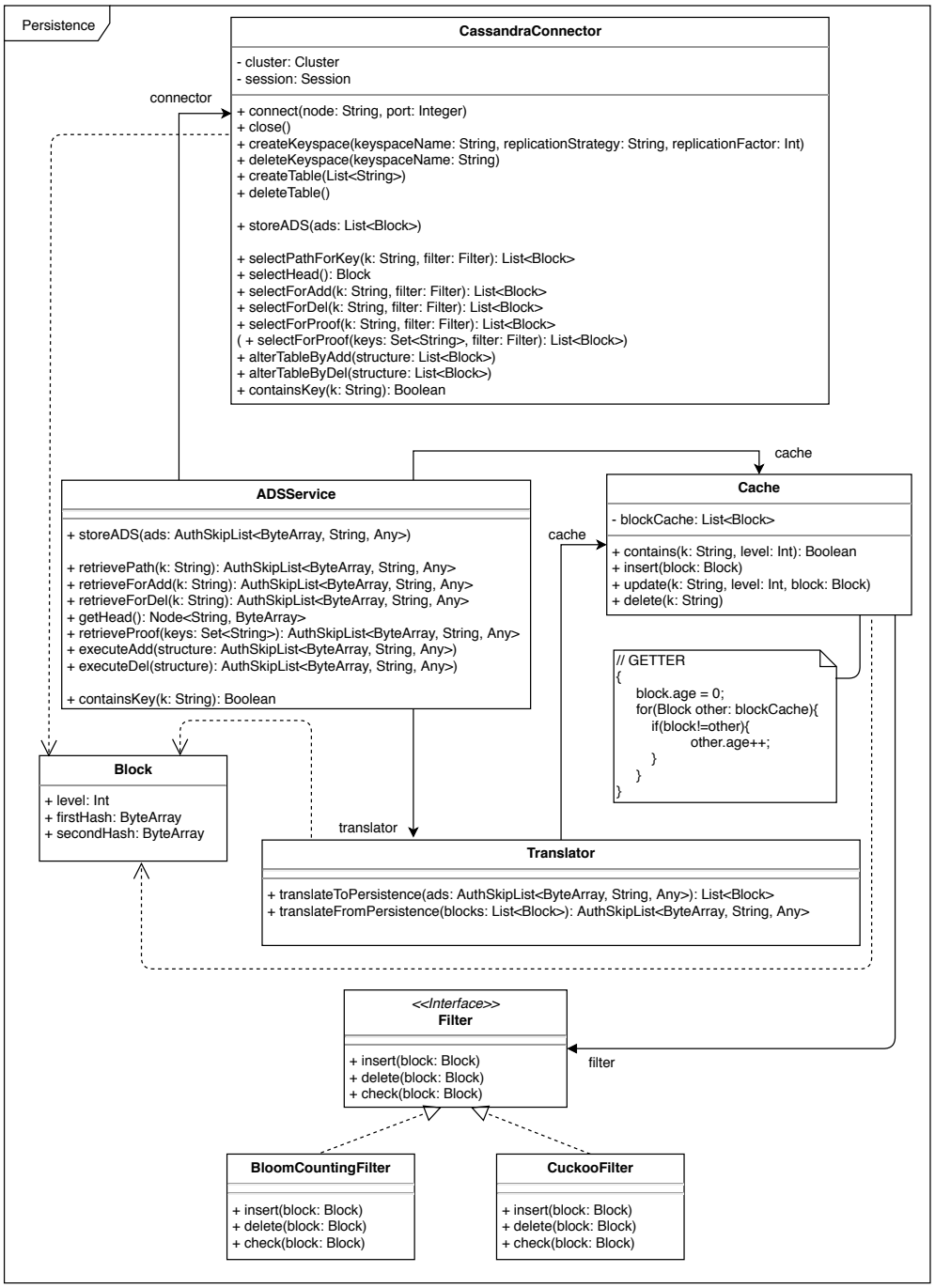


Figura 2.3: Diagramma delle classi di progetto per la parte di persistenza.

necessaria per permettere di ottenere la Proof relativa ad una specifica coppia chiave-valore (k, v).

getProof(keys: Set<K>): Proof<H, K, V> Restituisce la Proof relativa alle chiavi contenute nell'insieme *keys*. Estende le possibilità di *getWithProof(k)*, restituendo in un solo round una Proof cumulativa per un insieme di chiavi *keys*, valida e utilizzabile per tutte le stesse.

getValue(k: K): V? Data una chiave k, restituisce il valore v associato ad essa. Questa operazione permette dunque le operazioni di lettura, *read(k)*, richieste dai casi d'uso.

add(k: K, v: V): H Crea un *delta* rispetto alla struttura corrente, il quale viene memorizzato. Il delta riguarda l'aggiunta o la modifica dell'elemento corrispondente alla chiave k. L'operazione non modifica la struttura corrente.

del(k: K): H Crea un *delta* rispetto alla struttura corrente, il quale viene memorizzato. Il delta riguarda la cancellazione degli elementi relativi alla chiave k L'operazione non modifica la struttura corrente.

applyDeltas(): ADS<H, K, V> Compatta tutti i vari delta relativi alla struttura corrente e li applica in ordine, restituendo una nuova ADS, con le modifiche applicate. Questa operazione permette all'ADS di avere la capacità di aggiornarsi rispetto a richieste di update in arrivo dai client.

rootHash(): H Calcola e restituisce il Root Hash della struttura corrente. Questa operazione permette di ottenere il *digest* dell'intera struttura, come specificato dai requisiti.

isEmpty(): Boolean Informa se l'ADS è vuoto o meno.

E' necessario sottolineare riguardo alla prima operazione fornita, che, per *Creator*, sara l'ADS stesso a generare una Proof per una o più chiavi, proprio in quanto è l'unica entità a conoscere le informazioni per farlo. A seguito della creazione, essa sarà un entità indipendente, in grado di aggiornarsi in parallelo rispetto agli update. I calcoli e le operazioni poi eseguite sulla Proof, sono indipendenti dall'ADS di partenza.

Di fondamentale importanza è il requisito relativo alla possibilità di utilizzo dell'ADS in ambito concorrente. Deve essere possibile, dunque, poter effettuare operazioni in parallelo sull'ADS, senza che ci siano conflitti dovuti alla concorrenza. Pertanto è stato deciso di applicare un principio tipico del paradigma di programmazione funzionale, secondo cui i valori non vengono trovati cambiando lo stato dell'ADS, ma costruendo nuovi stati a partire dai precedenti. Per questo motivo le operazioni di scrittura, ovvero quelle che modificherebbero lo stato della struttura, lo fanno in maniera totalmente compatibile con quanto detto. Una spiegazione più dettagliata di come questo sia stato realizzato è posticipata in paragrafi successivi.

La scelta invece di disaccoppiare il momento dell'individuazione della operazioni di aggiunta, modifica o cancellazione, dal momento della loro effettiva applicazione, è da inquadrare in un contesto più ampio di utilizzo della libreria. Intuitivamente, questa funzionalità è fornita per permettere di gestire meglio gli stati in cui si può trovare la struttura stessa: ad esempio uno stato in cui può ricevere e accumulare update, piuttosto che uno stato in cui questi vengono applicati in blocco. Migliore è di conseguenza la gestione delle operazioni di lettura, in quanto per garantirne la consistenza, esse possono essere collocate in istanti temporali diversi, prima o dopo determinati stati della struttura.

L'interfaccia, e dunque la classi che implementeranno quest'ultima, deve implementare a sua volta l'interfaccia *Serializable*, in quanto dovrà avere la caratteristica di essere, appunto, serializzabile. Questo è motivato dal fatto che implementazioni di questa interfaccia saranno inserite all'interno di messaggi che dovranno essere spediti su rete, nell'ambito dell'utilizzo di questa API.

2.1.2 Interfaccia Proof

L'interfaccia della Proof, è stata resa parametrica rispetto a tre valori:

- **H**: il tipo dei vari *hash* presenti all'interno dei nodi della Proof.
- **K**: il tipo delle *chiavi* dei nodi.
- **V**: il tipo dei valori presenti nello *storage*.

Questa scelta rende l'interfaccia applicabile parametricamente rispetto alla scelta dei suddetti tipi, caratteristica utile per garantire flessibilità.

Le operazioni fornite dalla Proof, sulla base di requisiti e casi d'uso già discussi in fase di analisi, sono:

keys: Set<K> Insieme contenente tutte le chiavi per cui è valida la Proof.

rootHash(): H Restituisce il Root Hash della Proof. Questa, assieme alla successiva, sono requisiti funzionali già discussi in fase di analisi.

rootHash(list: List<Pair<K, V>>): Proof<H, K, V> Applica gli aggiornamenti specificati nella lista di coppie chiave-valore *list* e successivamente calcola e restituisce il nuovo Root Hash.

union(proof: Proof<H, K, V>): Proof<H, K, V> Effettua il *merge* di due Proof relative allo stesso istante temporale della stessa struttura da cui sono state generate. Restituisce una nuova Proof, cumulativa rispetto a quelle di partenza.

Di fondamentale importanza è il requisito relativo alla possibilità di utilizzo della Proof in ambito concorrente. Deve essere possibile, dunque, poter operare operazioni in parallelo sulla Proof, senza che ci siano conflitti dovuti alla concorrenza. Pertanto è stato deciso di applicare un principio tipico del paradigma di programmazione funzionale, secondo cui i valori non vengono trovati cambiando lo stato della Proof stessa, ma costruendo nuovi stati a partire dai precedenti. Per questo motivo le operazioni di scrittura, ovvero quelle che modificherebbero lo stato della struttura, lo fanno in maniera totalmente compatibile con quanto detto. Una spiegazione più dettagliata di come questo sia stato realizzato è posticipata in paragrafi successivi.

L'interfaccia, e dunque la classi che implementeranno quest'ultima, deve implementare a sua volta l'interfaccia *Serializable*, in quanto dovrà avere la caratteristica di essere, appunto, serializzabile. Questo è motivato dal fatto che implementazioni di questa interfaccia saranno inserite all'interno di messaggi che dovranno essere spediti su rete, nell'ambito dell'utilizzo di questa API.

2.1.3 Nodo e schema hashing

L'elemento base costituente sia della Skip List Autenticata sia della Skip List Proof, è il *nodo*. Esso, oltre ad essere a conoscenza dei suoi nodi adiacenti, nelle quattro direzioni, *next*, *prev*, *up* e *down*, è formato da tre campi interni:

- `key`. Un attributo chiave, necessario, per la struttura in cui è collocato, a mantenere l'ordinamento. Il tipo K della chiave deve dunque necessariamente implementare l'interfaccia *Comparable<K>*.
- `firstHash`. Un campo che contiene l'hash relativo al nodo immediatamente inferiore, *down*, o l'hash del valore dello storage puntato da questo nodo.
- `secondHash`. Un campo che contiene l'hash relativo al nodo successivo, *next*, se quest'ultimo è presente.

La scelta di tripartire all'interno del nodo le sezioni sopra menzionate, è stata presa sia per poter rispettare nella maniera più comoda e flessibile lo schema di hashing già discusso in fase di analisi, e sia per ridurre la quantità di operazioni per il refresh di tutta la struttura, a seguito di operazioni di update. Infatti a seguito di una operazione di inserimento, modifica o cancellazione, sarà necessario aggiornare solamente i campi strettamente interessati.

Il campo *down*, si precisa, che può essere sia una nodo, nella maggior parte dei casi, ma anche il valore vero e proprio, presente nello storage classico, legato a questo nodo. Sarà dunque solo ed esclusivamente questo nodo a conoscere questo valore e a memorizzare il suo hash nel campo *firstHash*.

La classe che implementa il nodo, deve implementare l'interfaccia *Serializable*, in quanto dovrà avere la caratteristica di essere, appunto, serializzabile. Questo è motivato dall'inserimento di oggetti di questo tipo all'interno di messaggi che dovranno essere spediti su rete, nell'ambito dell'utilizzo di questa API.

Lo schema di hashing presente all'interno della struttura, rispecchia quello già presentato in fase di analisi, e si adatta perfettamente alla separazione dei valori di hash attuata all'interno del nodo. Infatti da un lato il campo *firstHash*, memorizzerà sempre l'hash relativo al campo *down* del nodo, sia esso un nodo o un valore vero e proprio, di

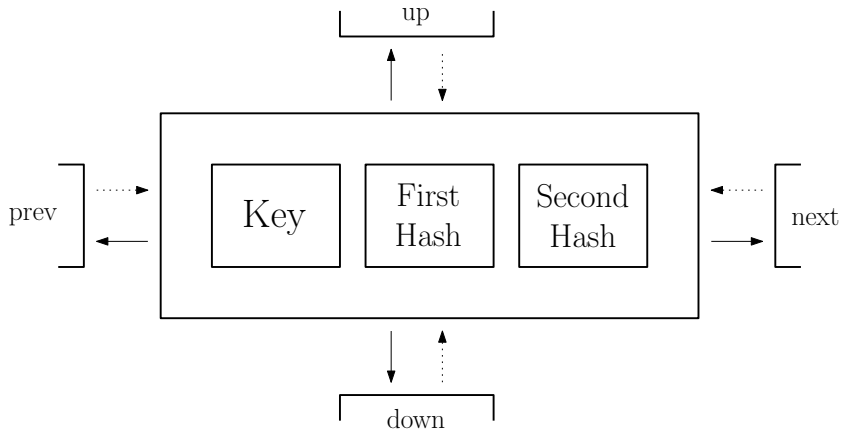


Figura 2.4: Esempio di un nodo.

tipo qualsiasi. Dall'altro lato il campo *secondHash* memorizzerà, solo se presente, l'hash relativo al campo *next* del nodo corrente. E' stato ritenuto conveniente eliminare, per i nodi sulla Base List, il fatto che il campo *secondHash*, nel caso in cui il nodo *next* sia un nodo *tower*, dovesse memorizzare l'hash relativo al valore sottostante il nodo *next*. Questo permette di continuare a garantire l'esecuzione di tutte le operazioni che comportino un refresh degli hash della struttura, in un tempo che sia con alta probabilità, $\mathcal{O}(logn)$, piuttosto che in tempo lineare. Inoltre, la proprietà di non-commutatività degli hash, si manifesta nel fatto che, per costruzione, se fosse invertito il valore presente nel campo *firstHash*, con quello presente nel campo *secondHash*, il cambiamento si ripercuoterebbe su tutti gli hash seguenti, fino ad arrivare alla modifica del Root Hash dell'intera struttura. Pertanto è fondamentale l'ordine e la posizione reciproca dei vari nodi.

Per quanto riguarda più specificatamente le funzioni crittografiche di hash, si è deciso di delegare il compito di calcolo di hash, singoli o concatenati, ad un Singleton, visibile a tutte le classi dell'API. Questa scelta ha permesso di rendere l'intera realizzazione totalmente flessibile e parametrica rispetto alla particolare funzione di hash che si volesse adottare, tra quelle disponibili elencate in fase di analisi. Risulta inoltre parametrico rispetto al tipo dell'hash stesso, sia esso una stringa di caratteri o un array di bytes.

2.1.4 Skip List Autenticata

Prima di addentrarsi dettagliatamente su aspetti di natura progettuale e algoritmica della struttura dati autenticata, è necessario fare delle considerazioni di carattere generale, conseguenza di quanto detto in fase di analisi. Prima di tutto si sottolinea l'inutilità, in questo studio progettuale, della colonna di nodi sentinella di tipo $+\infty$, la cui presenza comporterebbe solo un inutile spreco di memoria. I nodi sentinella di tipo $-\infty$, d'altro canto, servono sia per poter avere una colonna che sia concettualmente antecedente a tutte le altre colonne presenti nella struttura e sia di supporto all'applicazione dello schema di hashing. Secondariamente si è ritenuto superfluo, e puramente formale, il mantenimento di un livello superiore a tutti gli altri livelli, contenente soltanto, agli estremi, i nodi sentinella. Questo aggiungerebbe solamente una passo in più agli algoritmi di esecuzione delle operazioni. Nel contesto più ampio di una successiva applicazione di memorizzazione in persistenza della struttura, il mantenimento di questo livello superiore complicherebbe soltanto la conversione dal formato ad oggetti a quello della base di dati.

La ricerca in tempo logaritmico, cosi come le operazioni di inserimento, cancellazione o modifica, sono permesse dalle chiavi dei nodi, che dunque saranno ordinate: $k_{-\infty} < k_1 < k_2 < \cdots < k_n < k_{+\infty}$.

Segue la descrizione delle operazioni fornite, con enfasi sulle caratteristiche algoritmiche:

getWithProof(key: String): Pair<Any?, Proof<ByteArray, String, Any>>

L'algoritmo alla base di questa funzione, può essere diviso in due parti. La prima parte effettua una semplice ricerca di k sulla base delle chiavi contenute nei nodi, costruendo contemporaneamente la pila, *stack*, di cui un esempio è raffigurato in figura 2.5. Questa funzione, come tutte le operazioni di read offerte dalla Skip List Autenticata, si appoggiano su una serie di metodi per la ricerca, che oltre a restituire il nodo cercato, riempono la pila con opportuni dati per ogni passo effettutato. La seconda parte sfrutta proprio il contenuto di questa pila, per poter costruire la *Proof* relativa alla chiave cercata k. Infatti il contenuto della pila, analizzato a ritroso rispetto all'ordine di riempimento, rappresenta il percorso,

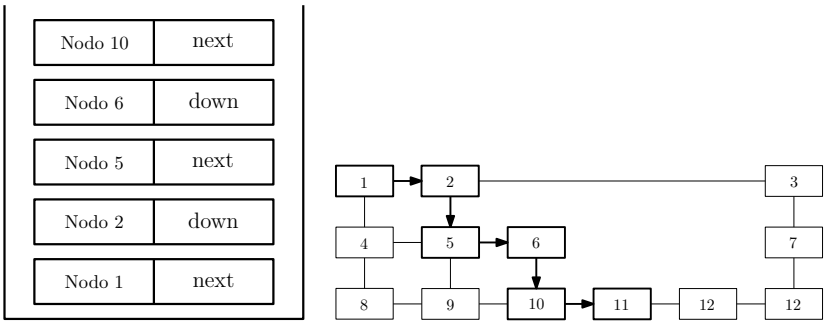


Figura 2.5: Esempio di una pila. I nodi sono stati volutamente segnati con un identificativo, per facilitare la comprensione dell'immagine, ma non hanno alcun riferimento con la progettazione o la realizzazione reale dei nodi. (a) La pila contenente in ordine di attraversamento, dal basso verso l'alto, i passi compiuti per comporre il *path*. I blocchi della pila sono composti dal nodo di partenza di ogni passo, affiancato dalla stringa che indica la direzione del passo compiuto. (b) Sono evidenziati, nella Skip List, i nodi e le direzioni presenti nello stack, a seguito della ricerca dell'elemento 10.

path, inverso, dal nodo allo *start node.*. Per costruire la Proof necessaria ad ottenere il Root Hash, bisogna però "integrare" i nodi presenti nello stack, proprio per costruzione dello schema di hash adottato sulla struttura. Se, ad esempio, un nodo avesse come *next* un *plateau*, allora esso dovrà essere inserito nella *Proof* in quanto sarà necessario nella computazione degli hash. Lo stesso vale per i nodi *down* di ogni nodo, che non siano presenti già nel *path*. In base a quanto detto, una volta effettuata la ricerca dell'elemento, che indirettamente costituisce la pila, viene ripercorso il *path* a ritroso partendo dall'elemento trovato, con opportune operazioni di pop. Ogni operazione di pop fornisce il successivo nodo presente nel *path* e la direzione, in senso inverso, che deve essere intrapresa per raggiungerlo. Tramite questa seconda informazione è possibile distinguere il caso in cui, seguendo il *path*, è necessario proseguire verso sinistra o verso l'alto. Nel primo caso sarà necessario integrare la *Proof* con il nodo *down* del successivo nodo del *path*. Nel secondo caso, invece l'integrazione avviene solo nel caso in cui il nodo *next* del successivo nodo del *path* sia un *plateau*.

getProof(keys: List<String>): Proof<ByteArray, String, Any> Questa fun-

zione in qualche modo estende la precedente, dando la possibilità, in un solo round, di richiedere ed ottenere una Proof valida per un insieme di chiavi passato come parametro. Ovviamente la lista passata dovrà necessariamente contenere solo chiavi presenti nella struttura. Per raggiungere questo obiettivo, da un lato la struttura si preoccupa di creare le singole Proof, una per volta, per ogni singola chiave passata come parametro, in quanto, come già detto, è l'unica entità che conosce le informazioni necessarie a farlo. Dall'altro è mantenuta una Proof cumulativa, valida per tutte le chiavi nell'intervallo $[k[0], k[corrente])$. Pertanto appena è creata una nuova Proof viene integrata con la Proof cumulativa corrente, richiedendo a quest'ultima una `union(proof: Proof<ByteArray, String, Any>)`. Questo procedimento prosegue fino all'avvenuta presa in considerazione dell'ultima chiave in *keys*, e termina con la restituzione della Proof cumulativa, a questo punta valida per tutto l'insieme di chiavi passato per parametro.

getValue(k: String): Any? Si tratta di una semplice operazione di *retrieve*. Infatti restituisce un valore di tipo V, associato alla chiave k passata come parametro. Se non è presente la chiave è restituito *null*.

applyDeltas(): ADS<ByteArray, String, Any> Questa operazione permette alla struttura di avere la capacità di aggiornarsi rispetto a richieste di update in arrivo dai client. Trattandosi di una operazione di scrittura, e dovendo innestarsi in un ambito di concorrenza già altrove menzionato, sarà necessario evitare di modificare direttamente lo stato della struttura. Dunque viene separato il momento della ricezione di particolari richieste di update dal momento della loro effettiva applicazione. In questo modo sarà possibile *clonare* la struttura corrente e applicare su quest'ultima gli update. Gli update possono tradursi in primitive di tipo `add(k: String, v: Any): ByteArray` nel caso si trattasse di aggiunte o modifiche, o di tipo `del(k: String): ByteArray` nel caso si trattasse di cancellazioni, andando a rappresentare dei *delta*, ossia delle variazioni in potenza della struttura. Essi sono accumulati, mantenendone l'ordine di "arrivo" in una pila, con opportune operazioni di *push*. A seguito dell'invocazione di `applyDeltas()`, le istanze della pila vengono valutata una ad una singolarmente, in ordine cronolo-

gico, con operazioni di *pop*, e applicate ad un clone della struttura corrente. Sarà poi questo, comprensivo degli update, ad essere restituito. L'attuazione effettiva delle varie azioni presenti all'interno della pila, è scissa, con l'obiettivo di migliorare la coesione generale, tra l'operazione `applyAdd(k: String, v: Any)` nel caso di aggiunte o modifiche, e l'operazione di `applyDel(k: String)` nel caso di cancellazioni. Queste scelte rendono invariato lo stato della struttura di partenza, che sarà dunque passibile di calcoli e computazioni anche contemporaneamente all'applicazione di update.

rootHash(): ByteArray Operazione che calcola e restituisce il Root Hash dell'intera struttura. Lo Start Node conterrà in ogni istante, nel campo *firstHash*, il contributo in termini di concatenzioni di hash crittografici dato da tutti gli elementi correlati al campo *down*, e nel campo *secondHash*, il contributo in termini di concatenazioni di hash crittografici dato da tutti gli elementi correlati al campo *next*. Viene, a questo punto, restituito l'hash derivato dalla concatenazione dei campi *firstHash* e *secondHash*.

isEmpty(): Boolean Questa operazione semplicemente restituisce *true* se la struttura non contiene nessuna coppia chiave-valore, *false* altrimenti.

clone(): ADS<ByteArray, String, Any> Copia la struttura e ne restituisce una identica, tranne per i delta che non vengono copiati. Il problema della clonazione è di non banale risoluzione. Infatti la Skip List Autenticata è a tutti gli effetti un grafo diretto ciclico. Una prima possibilità potrebbe consistere nel clonare ogni torre iterando sulla Base List, per poi preoccuparsi solo in un secondo momento dei collegamenti orizzontali. La struttura della Skip List però comporterebbe una scansione di tutte queste torri clonate per ogni nodo di ogni torre, operazione con costo computazionale molto elevato e non auspicabile. Richiederebbe inoltre la conoscenza, diretta o indiretta, dell'altezza di ogni singola torre. Per motivi analoghi, è da escludere anche la soluzione simmetrica, che memorizza ogni livello orizzontale, per poi preoccuparsi solo in un secondo momento dei collegamenti verticali. La soluzione poi effettivamente adottata, consiste nell'utilizzo di *Hash Map* di supporto, una per ogni livello della Skip List Autenticata, tutte di lunghez-

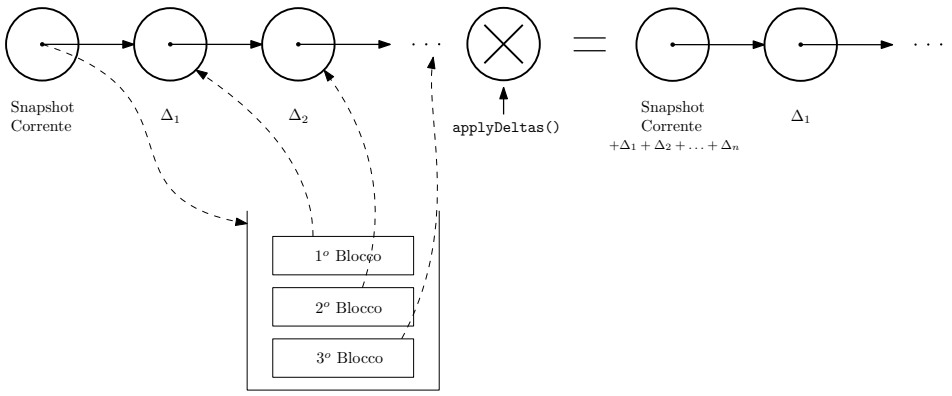


Figura 2.6: Schema che mostra il funzionamento dei delta. Tramite primitive di aggiunta e cancellazione è popolata la pula della struttura con dei blocchi, ordinati cronologicamente. L'uso successivo di *applyDeltas()* applica in ordine questi delta e restituisce un'altra struttura, con stato di partenza equivalente al primo *snapshot*, ma con tutti i Δ_n applicati. La pila viene quindi svuotata, ed è dunque poi pronta ad accogliere nuovi delta.

za pari alla dimensione della *base list* della struttura e tutte contenenti *entries* con tutte le chiavi della struttura. Iterando orizzontalmente su ogni livello, le Hash Map corrispondenti saranno riempite. A seconda del nodo scansionato, verrà aggiunto un riferimento a quest'ultimo nella *entry* corrispondente nella Hash Map. Se per una determinata chiave non sarà presente il nodo corrispondente a quel determinato livello, verrà mantenuto il valore di quella *entry* a null. Così facendo sarà possibile avere tutti i riferimenti ad ogni nodo, e allo stesso tempo conoscerne l'esatta collocazione reciproca. Sarà possibile poi clonare ogni singolo nodo, e sostituire il riferimento contenuto nei vari valori delle Hash Map con i vari cloni. A questo punto sarà necessario solo provvedere ai collegamenti, sia orizzontali che verticali. Per quanto riguarda i collegamenti verticali sarà sufficiente collegare tutti i nodi con stessa chiave su Hash Map corrispondenti a livelli adiacenti. Per quanto riguarda i collegamenti orizzontali sarà necessario collegare i nodi adiacenti all'interno di una stessa Hash Map, intervallati cioè da sole *entries* con valori nulli.

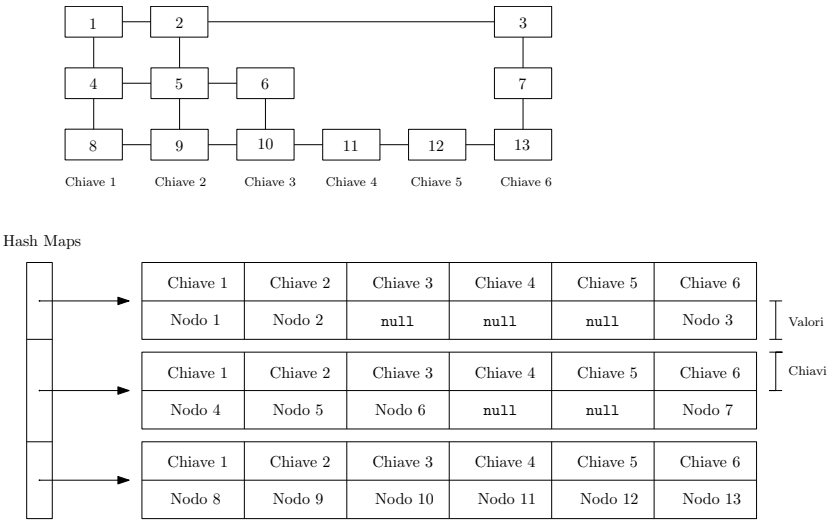


Figura 2.7: Immagine che illustra un esempio di clonazione vista nella sua prima fase, rispetto alla Skip List nell'esempio. E' possibile poi da questo stato clonare tutti i nodi e provvedere alla sistemazione di tutti i collegamenti.

2.1.5 Skip List Proof

Prima di addentrarsi dettagliatamente su aspetti di natura progettuale e algoritmica della Skip List Proof, è necessario fare delle premesse. La Proof, come già detto, concettualmente parlando, è relativa ad un particolare istante temporale della struttura dati autenticata da cui è stata generata. Dunque dal momento in cui inizia ad esistere, acquisisce uno stato e questo sarà completamente indipendente dallo stato della struttura di partenza. Pertanto la Skip List Proof, sarà una struttura dati indipendente e parallela all'ADS di partenza, di cui, inoltre, non avrà nessuna conoscenza, per motivi in parte già comprensibili, e in parte motivati in seguito nella sezione dedicata alla persistenza.

In base all'utilizzo dell'API, inoltre, nell'ambito del protocollo di integrità su dati nel Cloud [FGP18], sarà necessario poter applicare update ricevuti dall'ADS, nella forma compatibile con la Proof, alla Proof stessa, come già detto indipendente dalla struttura da cui è stata generata.

Fatti questi ragionamenti di natura concettuale, ora si riporta il ragionamento alla base di alcune decisioni progettuali e algoritmiche. La Proof per una chiave, rispetto alla Skip List Autenticata da cui è generata, è a tutti gli effetti poco più di un percorso,

path, dal nodo contenente quella specifica chiave, fino allo Start Node. Gli elementi che ha in più sono i nodi necessari a ricostruire tutto il percorso di concatenazioni di hash necessarie a calcolare e restituire il *Root Hash*, e sono o *plateau* presenti nel campo *next* di un nodo, non già presenti nel path, o nodi presenti nel campo *down* di un nodo, non già presenti nel path. La Proof cumulativa rispetto a un insieme di chiavi non fa altro che estendere la struttura corrente con ulteriori "path". Si potrebbe dunque pensare di rappresentare la Proof come fosse un albero binario, dove per ogni nodo, al posto di avere figlio destro e sinistro, si hanno rispettivamente nodo *next* e nodo *down*. Tuttavia è facile dimostrare che la struttura sarebbe fortemente sbilanciata e risulterebbe oneroso computazionalmente, tra le varie foglie che si verrebbero a formare, trovare quali sono le chiavi per cui la Proof è valida, a meno che non si utilizzi una struttura dati di supporto per memorizzare i riferimenti a tali nodi. Rimarrebbe tuttavia non ottimale per quanto riguarda l'applicazione di aggiornamenti alla struttura.

Si è preferito dunque utilizzare una struttura dati *ad hoc*, con forti somiglianze con la Skip List Autenticata. I nodi e l'organizzazione interna saranno dunque uguali alla controparte. Le differenze principali risiedono nell'utilizzo di molti meno puntatori inter-nodo e nell'assenza di qualsiasi riferimento ai valori dello storage classico, prerogativa esclusiva dell'ADS. I nodi corrispondenti alla Base List avranno conoscenza solo dell'hash di questi valori. In questo modo inoltre, molte delle proprietà caratteristiche della Skip List, sarebbero ereditate da questa struttura, chiamata per ovvi motivi, *Skip List Proof*, da cui il titolo del paragrafo.

Riguardo la gestione interna della struttura, si individuano due principali alternative. La prima consiste nell'utilizzare come punto di partenza per ogni operazione il nodo *HEAD*, corrispondente allo Start Node della Skip List Autenticata. La seconda invece consiste nell'avere una struttura contenente i riferimenti ai nodi contenenti le coppie chiave-valore per le quali è valida la Proof. La seconda soluzione presenta dei vantaggi soprattutto per quanto riguarda le operazioni che devono essere effettuate su specifiche chiavi, in quanto sarebbe possibile sostituire l'operazione di ricerca vera e propria, con un veloce accesso indicizzato. Tuttavia, come soluzione, risulterebbe svantaggiosa per tutti i calcoli che riguarderebbero il nodo *HEAD*, in quanto non sarebbe presente un riferimento fisso a quest'ultimo. La soluzione poi real-

mente adottata è un ibrido tra le due. Sarà dunque utilizzato il riferimento al nodo *HEAD* per le operazioni `rootHash()` e `union(proof: Proof<ByteArray, String, Any>): Proof<ByteArray, String, Any>` e la struttura proposta nella seconda soluzione, per ridurre di un fattore $\mathcal{O}(m)$ la complessità computazionale dell'operazione `rootHash(list: List<Pair<String, Any?>>)`, la quale nel caso di adozione esclusiva della prima soluzione sarebbe dell'ordine di $\mathcal{O}(m^2 * n)$, dove m è il numero di nodi nel *path* di ogni singola Proof, e n è il numero di chiavi per cui è valida la Proof stessa.

Segue la descrizione delle operazioni fornite, con enfasi sulle caratteristiche algoritmiche:

rootHash(): ByteArray L'operazione è analoga a quella della Skip List Autenticata appena descritta. Calcola e restituisce il Root Hash dell'intera proof. Il nodo *HEAD* conterrà in ogni istante, nel campo *firstHash*, il contributo in termini di concatenzioni di hash crittografici dato da tutti gli elementi correlati al campo *down*, e nel campo *secondHash*, il contributo in termini di concatenazioni di hash crittografici dato da tutti gli elementi correlati al campo *next*. Viene, a questo punto, restituito l'hash derivato dalla concatenazione dei campi *firstHash* e *secondHash*.

rootHash(list: List<Pair<String, Any?>>): ByteArray Nel caso in cui tutti i valori passati siano nulli, l'operazione risulta equivalente a `rootHash(): H`. Tramite la presenza di anche un solo valore non nullo, l'operazione diventerà di scrittura. Trattandosi di una operazione di scrittura, e dovendo innestarsi in un ambito di concorrenza già altrove menzionato, sarà necessario evitare di modificare direttamente lo stato della struttura. Dunque la struttura sarà clonata internamente e le modifiche agli hash saranno applicate sul clone. La struttura iniziale sarà dunque passibile di calcoli e computazioni anche contemporaneamente all'applicazione di queste modifiche. Il parametro che accetta è una lista di coppie chiave-valore, struttura che verrà processata iterativamente, rendendo superflua l'applicazione di una mappa di coppia chiave-valore. Nello specifico l'operazione processerà, per ogni chiave presente nella lista *list*, il nodo corrispondente nella Proof, aggiornando gli hash, risalendo fino al nodo *HEAD*. Al termine di questo

elenco di queste operazioni, la funzione restituirà la nuova Proof, con le modifiche applicate e il nuovo Root Hash.

union(proof: Proof<ByteArray, String, Any>): Proof<ByteArray, String, Any>

Trattandosi di una operazione di scrittura, e dovendo innestarsi in un ambito di concorrenza già altrove menzionato, sarà necessario evitare di modificare direttamente lo stato della struttura. Dunque la struttura sarà clonata internamente e l'unione verrà effettuta sul clone. L'unione consiste, intuitivamente, nell'integrazione della Proof su cui viene invocato il metodo, *this*, con la Proof passata per parametro, *proof*. Inizia, infatti, col trovare, tramite differenza tra insiemi, le chiavi presenti in *proof* che non sono presenti in *this*. Per ognuna di queste chiavi risultato della differenza, se presenti, viene effettuata una iterazione parallela a partire dall'HEAD di *this* e dall'HEAD di *proof*, che ha come fine quello di aggiungere a *this* i nodi per renderla valida per le ulteriori chiavi, contributo

di *proof*. E' mostrato in maniera più chiara nel seguente pseudocodice:

```
foreach key in (proof.keys - this.keys) do
    first-iterator = this.head;
    second-iterator = proof.head;
    if second-iterator.next != null then
        first-iterator.next = second-iterator.next;
    end
    if second-iterator.down != null then
        first-iterator.down = second-iterator.down;
    end
    if second-iterator.next.key ≤ key then
        first-iterator = first-iterator.next;
        second-iterator = second-iterator.next;
    else
        first-iterator = first-iterator.down;
        second-iterator = second-iterator.down;
    end
end
```

clone(): Proof<ByteArray, String, Any> Copia la struttura e ne restituisce una identica. La strategia utilizzata consiste nel clonare tutti i *path*, uno ad uno, e successivamente creare la Proof cumulativa tramite `union`. Dunque la complessità dovuta alla gestione del nodo *HEAD*, in comune per tutti i path, e alla gestione dei "sotto-percorsi", ovvero insiemi di nodi comuni a più path, è spostata proprio su quel metodo. L'algoritmo alla base della clonazione di un path, consiste nel "risalire", con una sequenza di passi verso l'alto o verso sinistra, da ogni nodo contenente chiavi per cui la Proof è valida, fino al nodo *HEAD* tenendo in considerazione l'eventuale presenza di nodi "fuori path" necessari allo schema di hashing, inseriti perché, come già detto, necessari allo schema di hashing. Se infatti, nella risalita è effettuato un passo verso l'alto, sarà necessario controllare se il nodo da clonare avesse un *plateau* nel campo *next*. Se invece è effettuato un

passo verso sinistra, bisognerà preoccuparsi di clonare l'eventuale nodo presente nel campo *down*.

2.2 Persistenza

Si passa ora alla sezione sulla *persistenza*. Prima di tutto è necessario sottolineare che i ragionamenti per questa sezione sono stati effettuati prendendo in considerazione un particolare DBMS non relazionale, Cassandra. Tuttavia, a meno di piccoli accorgimenti, lo studio teorico che è di seguito condotto, è altamente flessibile rispetto alla tipologia di database NoSQL scelta. Da come è deducibile dal Class Diagram in figura 2.3, il progetto per il servizio di persistenza è stato pensato come una collaborazione attiva tra più parti, per mantenere alta la coesione globale. Il nucleo centrale è rappresentato dalla classe Pure Fabrication *ADSService*. Questa funge da Controller per tutti i casi d'uso che necessitino di interagire con il database. Essa collabora con la classe *CassandraConnector*, la quale interagisce direttamente con il database, e con *Translator*, una classe di supporto per la conversione di formato dei dati scambiati con il database. E' previsto inoltre un sistema di *Cache*, rappresentato dalla classe omonima, per massimizzare l'efficienza nello scambio di dati con il database.

Nei successivi paragrafi si spiegherà più nel dettaglio ogni parte attiva all'interno del progetto di persistenza, illustrandone e motivandone tutte le scelte progettuali rilevanti.

2.2.1 Cassandra e NoSQL

Cassandra è un *Database Management System* non relazionale e distribuito con licenza *open source* e ottimizzato per la gestione di grandi quantità di dati. Fa parte della categoria di database NoSQL, ovvero di quei database dove la persistenza dei dati segue un meccanismo sostanzialmente diverso da quello appartenente al modello relazionale. Tuttavia, rispetto allo standard dei NoSQL, garantisce consistenza e supporta il sistema di transazioni, entrambe caratteristiche molto importanti nell'ambito della memorizzazione in persistenza della struttura dati autenticata. Le componenti principali di Cassandra, di interesse per questa introduzione, sono le seguenti:

Nodo Luogo dove sono memorizzati i dati.

Centro Dati Insieme di nodi correlati.

Cluster Componente che contiene uno o più centri dati.

Si tratta di un sistema *masterless*, ovvero tutti i nodi all'interno di un Cluster sono organizzati secondo uno schema ad anello e svolgono lo stesso ruolo, ovvero sono identici e contengono ognuno una replica dei dati. Ognuno di essi può dunque rispondere alle richieste di lettura e/o scrittura, al pari degli altri. Questo garantisce l'assenza di un *single point of failure*. Allo stesso tempo però tutti i nodi all'interno di un cluster sono interconnessi, in base al *Gossip Protocol*, garantendo la continuazione di servizio nel caso in cui uno o più nodi dovessero essere fuori uso.

Dal punto di vista logico, l'elemento più esterno è il *Keyspace*. Esso contiene liste di *Column Families*, ed è per questo che Cassandra è detto *column-oriented*. Una column family, a sua volta, è una collezione di righe, rappresentate da una *Map<RowKey, SortedMap<ColumnKey, ColumnValue>>*, dove però ogni riga è di lunghezza variabile. Mentre nel modello relazionale uno schema è fissato a priori, e una volta definite le colonne per una tabella, per ogni riga dovrò avere lo stesso numero di elementi, Cassandra è *schema-free*. Pertanto ogni riga potrà avere un numero variabile di colonne, modificabile nel tempo in base alle esigenze, offrendo altissimi livelli di flessibilità. Cassandra dunque non obbliga ogni riga ad avere tutte le colonne.
Questi sono gli aspetti necessari alla comprensione degli studi teorici che seguono, sebbene non coprano esaustivamente tutte le potenzialità di Cassandra.

2.2.2 Studi teorici di progettazione

In base a quanto detto si analizza ora il razionale alla base della memorizzazione della Skip List Autenticata in persistenza. Bisogna subito dire che risulta necessario trovare una corrispondenza tra il mondo ad oggetti, in-memory, dove deve essere utilizzata la struttura dati, e il mondo in persistenza, dove questa è effettivamente memorizzata. Le due parti hanno differenti modalità di rappresentazione interna degli stessi elementi. Ad esempio un nodo, o una serie di nodi della struttura, possono tradursi in una serie di stringhe di caratteri nel database. Pertanto per stabilire questa corrispondenza è necessario poter convertire da un formato "ad oggetti" al formato compatibile col da-

tabase e viceversa. Gli algoritmi alla base di questa conversione, tuttavia, variano a seconda di come vengono fatti corrispondere elementi in memoria, ossia insiemi di nodi, con elementi di Cassandra, da qui in poi per comodità definiti *blocchi*. Ossia dipendono strettamente dalla tipologia di rappresentazione logica scelta per rappresentare la struttura dati.

Per esempio, una possibile strategia, prevede il raggruppamento di nodi, a comporre blocchi in memoria, in maniera verticale. Ossia viene fatta corrispondere ad ogni torre della Skip List Autenticata un blocco in memoria. Questo, in ambiente Cassandra, corrisponderebbe ad avere per ogni torre una riga di lunghezza pari al numero di elementi presenti su quella torre. Questo è reso possibile, come detto, dal fatto che Cassandra non impone vincoli sul numero di colonne per ogni riga. Il problema principale di questo sistema risiede pero nel fatto che, supponendo di necessitare della coppia chiave-valore presente nel nodo sulla Base List memorizzato nel blocco i-esimo, obbliga al caricamento e/o restituzione di almeno tutti i blocchi dal primo fino all'i-esimo[3]. Altrimenti, nell'ottica del fatto che potenzialmente ogni blocco in memoria potrebbe corrispondere ad una query sia in lettura che in scrittura, è ragionevole pensare di voler ridurne il numero, aumentando il passo del blocco stesso. Dunque ogni elemento piuttosto che contenere una sola torre, potrebbe contenerne due o più. Questa soluzione tuttavia sebbene riduca il numero medio di query necessarie, dipendentemente dalla larghezza scelta per il passo, presenta sostanzialmente gli stessi problemi caratteristici di una schematizzazione verticale, la quale poco si confà alla natura della Skip List Autenticata, dal momento che elementi in alto a sinistra nella struttura saranno sempre o quasi sempre interessati dalle query. Con la memorizzazione verticale sarò obbligato sempre a restituire tutte le torri, anche se dovessero servire solo pochi singoli nodi.

Un'altra possibile strategia, invece, prevederebbe il raggruppamento di nodi secondo uno schema orizzontale. In questo caso, è opportuno anticipare che per definizione, la Skip List Autenticata, tende a crescere molto di più orizzontalmente che verticalmente. In particolare, sia n il numero di chiavi contenute, la dimensione orizzontale crescerà linearmente con l'aumento di n, mentre l'altezza seguirà un trend logaritmico. Dunque, per motivi deducibili, non avrebbe senso avere blocchi orizzontali che ripercorrano l'in-

[3]Potrebbe essere necessario richiedere anche blocchi successivi all'i-esimo in quanto potrebbero memorizzare plateau fondamentali per ricostruire lo schema di hashing.

tera lunghezza, ma essi dovranno essere segmentati. La segmentazione potrebbe essere simmetrica, ovvero i blocchi, di larghezza variabile ma fissabile a priori tramite limiti massimo e minimo di chiavi contenibili al suo interno, sono tutti incolonnati attraverso i vari livelli. Sebbene questa rappresentazione presenti interessanti proprietà probabilistiche, ha come principale difetto quello di non mantenere costante il numero di elementi per blocco. Il modo per ovviare a ciò è l'adozione di una strategia di segmentazione asimmetrica, dove quindi i blocchi avranno tutti la stessa larghezza, imponibile a priori, ma perderanno la possibilità di essere incolonnati. Per la strutturazione iniziale si può pensare che, poiché con alta probabilità ogni livello L_i avrà il doppio del numero degli elementi del livello L_{i+1}, allora questa corrispondenza può riflettersi sui blocchi. Ed in particolare ogni blocco avrà un limite massimo e minimo e, almeno inizialmente, avrà in corrispondenza nel livello inferiore, esattamente due blocchi[4], i cui limiti massimo e minimo, divideranno esattamente il range del blocco sovrastante. In questo modo, lo schema dei blocchi, assomiglierà ad un albero binario. Questa struttura riesce a rendere tendenzialmente costante la dimensione del blocco, ma presenta dei problemi laddove i nodi, aggiunti randomicamente, formino una struttura molto discostata da quella dell'albero binario. Inoltre la asimmetria dello schema si andrebbe sempre più ad inacerbare nel caso di scritture che vanno ad aggiungere/rimuovere blocchi, o cambiarne i limiti massimo e/o minimo, complicandone la gestione dal punto di vista progettuale.

Si osservi che un tratto comune nelle soluzioni sopra proposte riguarda la scelta della dimensione del blocco, ovvero il numero di elementi contenuti, e dunque per costruzione la larghezza. Si tratterebbe di un *trade-off* dove da un lato al crescere del blocco, cresce l'entità del traffico dati di ogni query e il numero di nodi inutili ai fini dell'esecuzione delle operazioni, mentre si riduce il numero di elementi da richiedere/caricare per operazione. Dall'altro un blocco più piccolo consentirebbe di raggiungere una grana più fine riducendo il numero di nodi inutili e l'entità del traffico dati per ogni query, richiedendo però più operazioni di restituzione/caricamento.

Sarebbe auspicabile poter minimizzare la dimensione degli elementi che transitano in modo da poter minimizzare il numero di elementi inutili. Questo è permesso dall'assegnare ad ogni nodo della struttura, un blocco cassandra a lui dedicato. In questo

[4]Assunto che la probabilità di Coin Flip sia pari ad $\frac{1}{2}$.

modo sarà possibile operare solo ed esclusivamente con i nodi interessati, trattando solo con elementi utili ai fini di una operazione da eseguire.

Questa soluzione tuttavia massimizza contestualmente il numero di elementi in transito per le query.

Di conseguenza sarebbe auspicabile rendere le query indipendenti l'una dall'altra. Fare in modo cioè che una query non dipendesse dal risultato di quella precedente. In questo modo esse potranno essere lanciate in parallelo in modo da permettere il caricamento o la restituzione di tutti gli elementi necessari, in un solo round.

Tuttavia, la richiesta di blocchi in maniera parallela richiede qualche informazione aggiuntiva. Una prima possibilità riguarda la conoscenza della struttura dei blocchi, e dunque dei nodi, ovvero la loro collocazione reciproca. Tuttavia questa è soggetta a continui cambiamenti nel caso di scritture che comportino aggiunte, cancellazioni o modifiche di blocchi. Pertanto la struttura andrebbe tenuta aggiornata o, in alternativa, richiesta e/o ricalcolata prima di ogni operazione. Questo in linea teorica permetterebbe di effettuare query in parallelo, ma piuttosto che avere un round per operazione, ne avrei due, dove uno sarebbe dedicato appunto all'ottenimento della struttura. Ragionando su quanto detto, si nota che il problema risiede nel fatto che si vorrebbe estrapolare un dato, di per se derivabile dai blocchi in qualsiasi istante, e materializzarlo lato client. Questa operazione comporta inevitabilmente tutte le difficoltà nel tenere questo dato aggiornato. L'ideale sarebbe il lasciare la gestione della struttura a chi può ottenerla e usarla, o in generale conoscerla, in maniera diretta, cioè lato server. La soluzione sta nello spostare la complessità di questa operazione tutta lato server, inviando solo i dati necessari a quest'ultimo e delegando il compito di individuare i nodi necessari e restituirli ad uno script server-side. Nel caso specifico di Cassandra è possibile affidarsi alle *User-Defined Function*, UDF, ovvero delle funzioni definite dall'utente, che possono essere applicate ai dati memorizzati in persistenza.

Entrando più nello specifico dell'utilizzo, ovvero delle operazioni da eseguire dell'API, si nota la compresenza di operazioni di scrittura e di lettura. Nel caso di blocchi contenenti più nodi, le operazioni di scrittura possono diventare molto complicate da gestire. Infatti una modifica può potenzialmente completamente stravolgere un blocco,

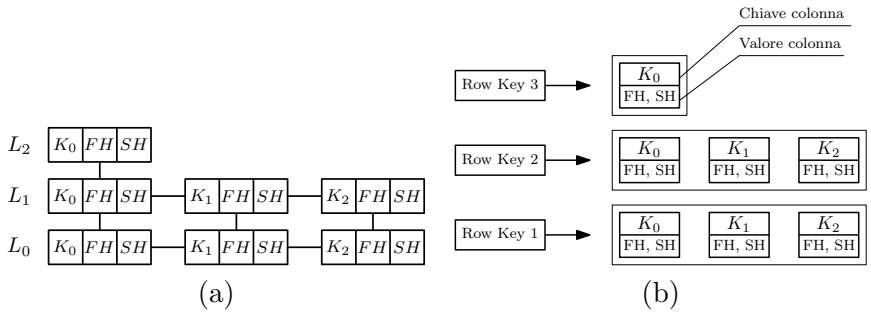


Figura 2.8: **(a)** Esempio di una Skip List Autenticata e **(b)** sua rispettiva possibile rappresentazione in persistenza su Cassandra. Ogni blocco, formato da una coppia chiave-valore di colonna, contiene le informazioni di un solo nodo.

sia nelle informazioni che contiene, sia nelle meta-informazioni[5], fino a rendere possibile cancellazioni o creazioni di nuovi blocchi. Far corrispondere un singolo nodo ad un blocco Cassandra, risolve questo problema riducendo drasticamente la complessità di gestione della struttura in persistenza.

Si ragiona ora sulle informazioni, ed eventualmente meta-informazioni, che dovrebbe contenere un blocco Cassandra, nel caso in cui ad ogni nodo corrisponda un blocco. Chiaramente dovrà contenere i due valori presenti nei campi *First Hash* e *Second Hash* cosi come la chiave. Potrebbe inoltre essere necessario memorizzare le informazione relative al livello in cui si trova il nodo, cosi come ai suoi campi *next*, *prev*, *up* e *down*. Anche se seguendo una opportuna collocazione dei blocchi in persistenza è possibile rendere superflua la presenza di queste informazioni nella maggior parte delle query, deducendole facilmente dalla sequenza di blocchi restituita, si rende necessaria la presenza dell'informazione riguardante il livello di struttura in cui si trovano. Così facendo ogni blocco sarà completamente identificabile.

2.2.3 Translator

La conversione tra formato ad oggetti, in memoria, a formato in persistenza e viceversa, nel progetto precedentemente illustrato è affidata al *Translator*.

[5]Si tratta di meta-dati come il numero di elementi contenuti, il limite massimo, il limite minimo, il livello a cui si trova il blocco, e cosi via.

Le operazioni che fornisce sono banalmente due, e sono simmetriche[6]:

translateToPersistence(ads: AuthSkipList<...>): List<Block> E' necessaria la conoscenza della logica di schematizzazione dei blocchi vigente all'interno del database. In questo modo è possibile preparare nell'esatta sequenza i blocchi che saranno poi caricati sul database.

translateFromPersistence(blocks: List<Block>): AuthSkipList<...> Anche in questo caso è necessaria la conoscenza della logica di schematizzazione dei blocchi vigente all'interno del database. Infatti è possibile dedurre dai blocchi ricevuti dalla query, le informazioni necessarie a ricostruire la sezione di Skip List Autenticata. Come già detto infatti, in base all'operazione invocata sulla Skip List Autenticata, il Sistema provvederà a restituire soltanto i nodi che saranno toccati dall'operazione, ovvero solo quelli strettamente necessari all'esecuzione di quell'operazione.

2.2.4 CassandraConnector

Il *CassandraConnector* ha il compito di stabilire la connessione vera a propria con il database, in base al particolare nodo e alla porta, entrambi specificati per parametro. Sarà dunque suo il compito di creare keyspace e tabelle necessarie alla memorizzazione della struttura dati, cosi come a fornire le primitive opportune per la gestione di operazioni di scrittura e lettura.

Le operazioni principali che fornisce sono:

connect(node: String, port: Integer) Stabilisce la connessione con il nodo *node* alla porta *port*. Se non specificata, verrà utilizzata quella di default 9042.

close() Chiude la connessione.

createKeyspace(name: String, strategy: String, factor: Int) Crea il Keyspace con nome $keyspaceName$, fattore di replicazione dato da $replicationFactor$ e

[6]Sottintesa la terna <ByteArray, String, Any> per i tipi della Skip List Autenticata.

strategia di replicazione data da *replicationStrategy*. Questi due parametri stabiliscono il numero di repliche dei dati sui vari nodi e la strategia di distribuzione delle stesse all'interno dell'anello.

createTable(List<String>) Crea la Column Family relativa alla struttura. Può accettare come parametro la lista di tutte le chiavi presenti nella struttura iniziale, che possono essere altrimenti aggiunte progressivamente.

deleteKeyspace(keyspaceName: String) Elimina il Keyspace di nome $keyspaceName$.

deleteTable() Elimina la Column Family relativa alla struttura.

storeADS(ads: List<Block>) Carica tutta la struttura passata per parametro. Funzione di utilità introdotta in base ad una specifica di utilizzo nell'ambito del protocollo, dove viene inizialmente creata una struttura iniziale, che dovrà dunque poter essere caricata in persistenza.

selectPathForKey(k: String, filter: Filter): List<Block> Restituisce la sequenza di blocchi costituenti il *path* dallo *Start Node* al nodo contenente la chiave k.

selectHead(): Block Restituisce i campi *First Hash* e *Second Hash* dello Start Node.

selectForAdd(k: String, filter: Filter): List<Block> Restituisce la sequenza di blocchi necessaria a poter eseguire un'inserimento o una modifica sulla struttura. Tiene conto dunque sia dell'eventualità di aggiunta di nuovi nodi, sia della modifica di nodi già presenti.

selectForDel(k: String, filter: Filter): List<Block> Restituisce la sequenza di blocchi necessaria a poter eseguire una cancellazione sulla struttura. Tiene conto dunque sia dei nodi che saranno rimossi, sia di quelli che subiranno una modifica a seguito della rimozione di una torre.

selectForProof(k: String, filter: Filter): List<Block> Restituisce la sequenza di blocchi necessaria per poter costruire la Proof relativa alla chiave k. Questa funzione potrebbe essere utilizzata sia singolarmente per ottenere una Proof valida

per una sola chiave, sia nell'ambito di una richiesta di una Proof cumulativa. In questo caso sarebbe necessario chiamare questa funzione tante volte quante sono le chiavi. E' possibile operare una ottimizzazione che restituisca la sequenza di blocchi necessari per poter ricostruire la Proof cumulativa per un insieme di chiavi. In tal caso la firma cambierebbe in: `selectForProof(keys: Set<String>)`.

alterTableByAdd(structure: List<Block>) Effettua la sovrascrittura su database dei blocchi modificati e la scrittura di blocchi eventualmente aggiunti. Le informazioni relative all'aggiunta devono essere opportunamente codificate all'interno di una stringa di caratteri, sotto forma di lista di blocchi, che sarà interpretata poi lato server.

alterTableByDel(structure: List<Block>) Effettua la sovrascrittura su database dei blocchi modificati e la cancellazione di quelli rimossi. Le informazioni relative all'aggiunta devono essere opportunamente codificate all'interno di una stringa di caratteri, sotto forma di lista di blocchi, che sarà interpretata poi lato server.

containsKey(k: String): Boolean . Restituisce true se la chiave è presente in struttura, false altrimenti.

2.2.5 ADSService

Come già deducibile da precedenti discussioni, l'*ADSService* è un centro-stella per le altre parti progettuali. Essa, in base alle operazioni invocate sulla Skip List Autenticata, si occupa della cooperazione tra Translator e Connector per fornire le informazioni necessarie all'esecuzione delle operazioni stesse, da un lato, e per tenere aggiornata la struttura in persistenza, dall'altro.

Le principali operazioni fornite sono:

storeADS(ads: AuthSkipList<ByteArray, String, Any>) Funzione che permette il caricamento su database dell'intera struttura passata per parametro. La conversione di formato è delegata al Translator, e successivamente la sequenza ottenuta è passata al Connector che dovrà caricarla in persistenza.

retrievePath(k: String): AuthSkipList<ByteArray, String, Any> Funzione che permette l'ottenimento della struttura necessaria a costituire un *path*. Utile dunque per permettere le operazioni di ricerca e verifica di contenimento di chiave.

retrieveForAdd(k: String): AuthSkipList<ByteArray, String, Any> Restituisce la struttura opportuna su cui potrà essere eseguita una modifica o una aggiunta sulla struttura.

retrieveForDel(k: String): AuthSkipList<ByteArray, String, Any> Restituisce la struttura opportuna su cui potrà essere eseguita una cancellazione sulla struttura.

getHead(): Node<String, ByteArray> Permette l'ottenimento dello Start Node. Utile dunque per tutte le operazioni della Skip List Autenticata che necessitano solo di quest'ultimo.

retrieveProof(keys: Set<String>): AuthSkipList<ByteArray, String, Any> Permette l'ottenimento della struttura da cui sarà possibile ricavare la Proof cumulativa per un insieme di chiavi *keys* passato per parametro.

executeAdd(structure: AuthSkipList<ByteArray, String, Any>) Permette l'aggiornamento del database a seguito di una operazione di aggiunta/modifica effettuata sulla struttura.

executeDel(structure: AuthSkipList<ByteArray, String, Any>) Permette l'aggiornamento del database a seguito di una operazione di cancellazione effettuata sulla struttura.

containsKey(k: String): Boolean Permette di verificare la presenza di una chiave nella struttura.

2.2.6 Comportamento asincrono

In base ai ragionamenti condotti fin qui, è possibile operare una forma di ottimizzazione generale, data dall'introduzione di parallelismo sull'esecuzione delle varie query e/o operazioni. Infatti potrebbero essere frequenti situazioni in cui sarebbe possibile fare

chiamate asincrone su varie funzioni la cui esecuzione parallela non creerebbe collisioni. Un esempio è il caso di letture parallele: le operazioni di lettura non modificano in alcun modo la struttura, pertanto possono essere distribuite su più thread di esecuzione paralleli. In questo caso le operazioni consisterebbero in una serie di chiamate asincrone annidate. Per gestirne opportunamente il flusso è necessario fare uso delle *promesse* di restituzione futura di un valore, per poter gestire il responso delle varie esecuzioni.

2.2.7 Caching

All'interno del progetto di persistenza è stato introdotto anche un sistema di *caching* dei blocchi. Si è ipotizzato di applicare una *Replacement policy* di tipo LRU, ovvero *Last Recently Used*. Separatamente per ciascun blocco in cache è associato un *replacement index*, ovvero un "contatore di età". I blocchi aventi il più alto valore saranno eliminati dalla cache, per fare spazio ad altri blocchi. Inoltre, ogni volta che un blocco di cache è acceduto il suo contatore è settato a zero, mentre gli altri contatori degli altri blocchi sono aumentati di 1, ad esclusione dei blocchi contenenti nodi direttamente adiacenti al blocco acceduto. La decisione del valore massimo di età, ovvero di una soglia massima di permanenza di un blocco in cache, è un valore variabile. All'aumentare della soglia, chiaramente, aumenteranno le probabilità di cache hit, a discapito di più spazio di memoria utilizzato. All'interno di questo progetto la cache viene riempita a seguito di una query di qualsiasi tipo. Le query che comportano scritture su database, devono preoccuparsi di aggiornare anche i blocchi in cache che hanno modificato. Tuttavia risulta necessario informare lo script server-side, in qualche modo, riguardo gli elementi già presenti in cache, che dunque non dovranno essere restituiti. Questo può essere risolto, aggiungendo alle query di tipo retrieve i blocchi già posseduti.

2.2.8 Bloom Filters

Cosi facendo tuttavia, le query in uscita dal client hanno una lunghezza che è di ordine $\mathcal{O}(n)$ rispetto ai blocchi posseduti in cache. Si può allora prendere in considerazione l'adozione di *Bloom Filters*, per passare informazioni al database riguardo la cache locale in $\mathcal{O}(1)$. Un Bloom Filter è una struttura dati probabilistica usata per testare se un elemento è o meno presente all'interno di un insieme. Sebbene siano presenti

falsi positivi, è esclusa la possibilità di falsi negativi. In altre parole, una richiesta può risultare in un "possibilmente presente" o in un "sicuramente assente". Questo ben si adatta al caso di studio della cache, in quanto è perfettamente accettabile il ricevere un numero esiguo di blocchi aggiuntivi, in realtà già presenti in cache. Non sarebbe invece stato accettabile il caso in cui sarebbero stati possibili falsi negativi, col rischio dunque di non ottenere blocchi invece necessari. Un Bloom Filter senza elementi è un array di m bit, tutti settati a 0. Possiede anche k differenti funzioni crittografiche di hash, ognuna delle quali "mappa" gli elementi dell'insieme a una delle m posizioni dell'array, generando una distribuzione randomica uniforme. Per aggiungere un elemento, bisogna farlo passare per le k funzioni di hash, per ottenere dunque k posizioni sull'array. I bit relativi a queste posizioni saranno dunque settati a 1. Dunque il test sulla presenza di un elemento segue lo stesso *iter*: se anche solo una delle posizioni ha il bit settato a 0, vorrà dire che l'elemento è assente, altrimenti è presente. La possibilità di falsi positivi è data dal fatto che uno o più bit settati a 1 rispetto ad un elemento i_1, potrebbero essere in realtà stati settati dall'inserimento di un altro elemento i_2, dove $i_1 \neq i_2$. Gli elementi dunque possono essere aggiunti all'insieme, ma non possono essere rimossi. Infatti una rimozione non è possibile, proprio dal fatto che falsi negativi non sono permessi. Settare a 0 anche solo un bit di un elemento, potrebbe comportare l'eliminazione di altri elementi mappati su quel bit. I Bloom Filter hanno inoltre l'importante proprietà, utile anch'essa ai fini dei questo studio di caso, secondo cui il tempo richiesto per aggiunte o per verificare la presenza di un elemento è costante, ed è $\mathcal{O}(k)$, ovvero è completamente indipendente dal numero di elementi nell'insieme. I k *lookup*, inoltre, sono indipendenti tra loro, e potrebbero essere parallelizzati per aumentare l'efficienza.

Dovendo tuttavia rispecchiare la cache presente in memoria, è necessaria la possibilità di cancellazione di un elemento, come detto impossibile per un semplice Bloom Filter. Sarebbe possibile l'utilizzo di una sua estensione, i *Counting Filters*, che aggiungono esattamente quella caratteristica, tramite la promozione di ogni posizione dell'array da 1-bit ad n-bit. In questo modo l'operazione di inserimento non farà altro che incrementare il valore presente nella posizione, e la verifica di presenza controllerà che nelle k posizioni non ci sia un bit pari a 0. Tuttavia, se già i Bloom Filter classici, utilizzano $1.44 \log 2\frac{1}{\epsilon}$ bit di spazio per elemento inserito nell'insieme, dove ϵ è la pro-

babilità di falso positivo della struttura, i Counting Filters utilizzano mediamende 3-4 volte lo stesso spazio. Questo potrebbe dunque ridurne l'utilità di utilizzo all'interno del progetto di persistenza. Tuttavia una soluzione ottimale invece prevede l'utilizzo dei *Cuckoo Filters* [BFM14], ovvero una evoluzione dei Bloom Filters, che permettono sia aggiunte che cancellazioni, ma ottimizzate per l'utilizzo dello spazio. Infatti possiedono un algoritmo di riempimento dell'array più sofisticato in grado di garantire prestazioni molto superiori a quelle di estensioni classiche dei Bloom Filters.

Dunque, in definitiva, il filtro passato nella query permetterà alla UDF server-side il check se un blocco è presente o meno in cache, e potrà dunque decidere se restituirlo o meno. In cache dunque sarà presente una lista di blocchi dove continuerà a valere quanto detto fin ora, accostata ad un Cuckoo Filter, sincronizzato con quest'ultima.

2.2.9 Modello con attori

Se si vuole inoltre considerare che, nell'utilizzo pratico, attori presenti sul server operano in parallelo sulla struttura dati, ovvero eseguono operazioni in maniera concorrente sulla stessa risorsa, è possibile introdurre una piccola estensione che si basa sull'utilizzo del modello ad attori. Il Modello ad Attori è una recente teoria di programmazione concorrente che consente la creazione di applicazioni sviluppate attorno a delle unità di processamento chiamate *Attori*, e dove il processamento degli attori è descrivibile come un ciclo indefinito di ricezione e processamento di messaggi, primitiva di comunicazione tra gli stessi. E' possibile dunque assegnare un attore per ogni nodo della struttura dati. Così facendo si potrà accedere alla struttura dati da vari punti in modo parallelo. L'attore radice, ovvero quello che corrisponderebbe allo Start Node, potrebbe rappresentare un collo di bottiglia. Tuttavia essendo un caso circoscritto, è possibile dedicargli potenza di calcolo esclusiva. Una miglioria a questo sistema potrebbe consistere nell'avere due attori per ogni nodo della struttura, uno dedicato alle letture e uno alle scritture. Dunque, una volta caricata la struttura in persistenza, viene creata una corrispondenza biunivoca tra i blocchi cassandra e una serie di attori, i quali conoscono solo il riferimento al nodo a loro assegnato, ed inoltre si conosce la struttura reciproca degli attori stessi. Dunque a questo punto, invece di utilizzare una forma di scripting server-side per l'individuazione, restituzione e scrittura dei nodi necessari passo per

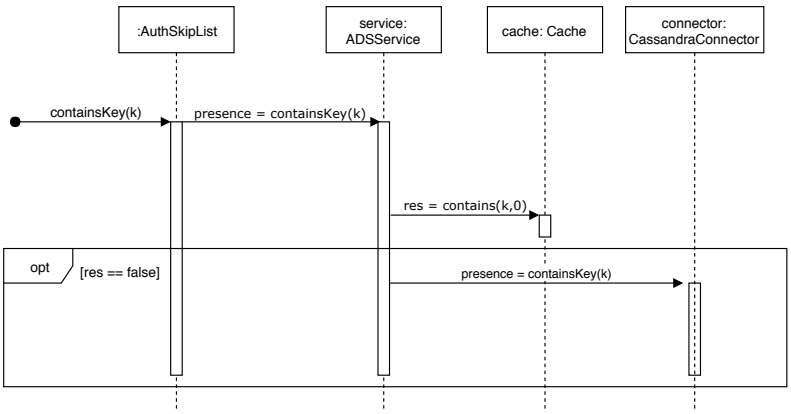


Figura 2.9: Diagramma di sequenza per l'operazione di check sulla presenza di una chiave.

questa rete di attori presente in memoria. Questo sistema consentirebbe inoltre di bypassare il sistema transazionale dei classici database, adattando le query in maniera ottimale alla natura della Skip List, che, come già detto in fase di analisi, è molto poco bloccante negli accessi paralleli. Infatti devono essere bloccati solo i blocchi interessati da una operazione di scrittura, mentre possono essere lasciati disponibili tutti gli altri blocchi. Inoltre, immaginando una sequenza di blocchi $B_0, B_1, ..., B_n$ da processare all'interno di una query, in questo modo è facile immaginare un sistema di *pipeline* in cui già dal momento in cui si sta processando il blocco B_1, si rende libero il blocco precedente, e cosi via.

2.2.10 Progettazione

Seguono delle immagini per illustrare, sulla base di quanto detto, la collaborazione tra le varie parti per permettere la memorizzazione della Skip List Autenticata in persistenza e contestualmente l'utilizzo dell'API. I diagrammi di sequenza sono costruiti in base alle operazioni eseguibili dalla Skip List Autenticata e mostrano come sia possibile rispettare il requisito della trasparenza di questa estensione rispetto alla libreria standard.[7]

[7]Si perdoni un piccolo abuso di notazione sui vincoli all'interno dei frame condizionali, attuato per favorire la comprensione del lettore.

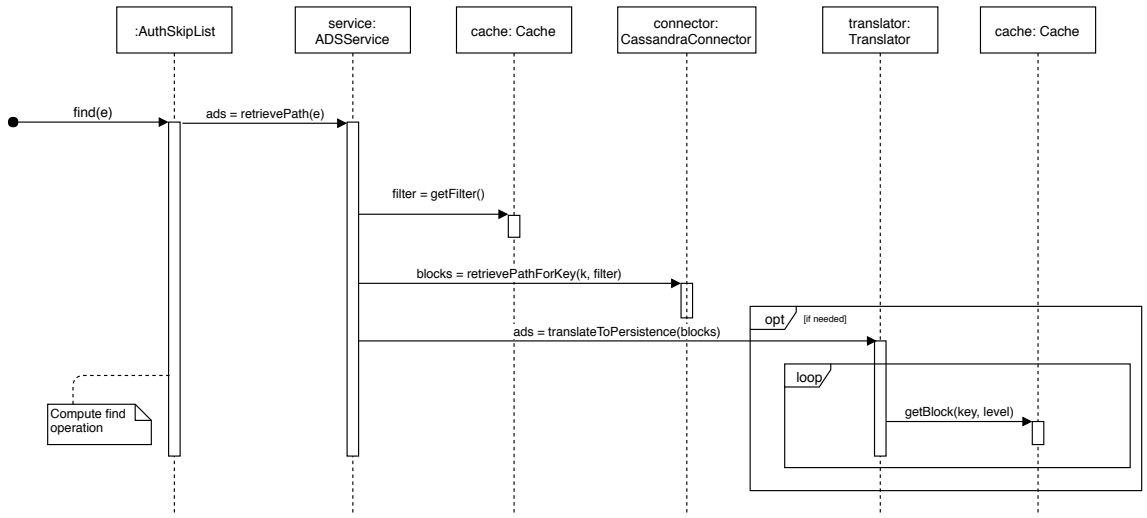


Figura 2.10: Diagramma di sequenza per l'operazione di ricerca di una chiave.

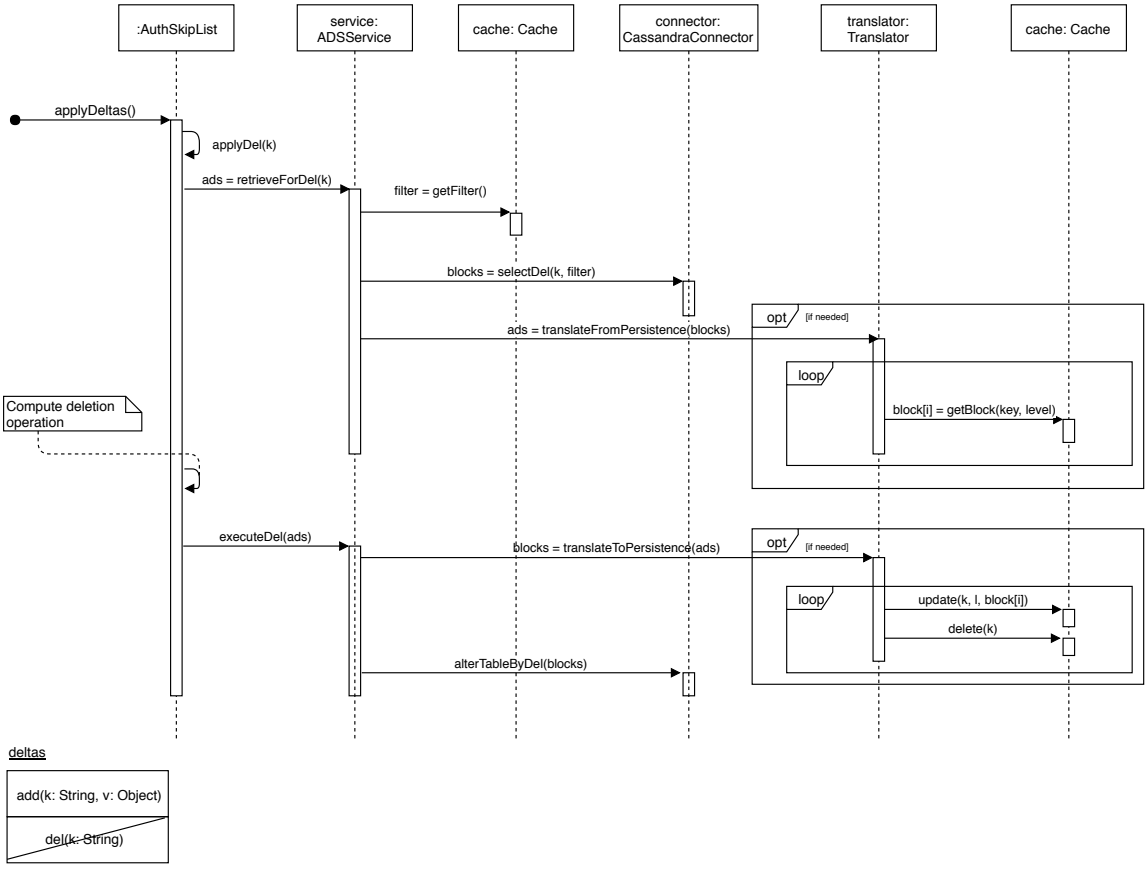


Figura 2.11: Diagramma di sequenza per l'operazione di applyDeltas(). Lo schema mostra un particolare esempio in cui devono essere effettuate due operazioni, una di cancellazione e una di aggiunta. E' mostrata solo la parte di cancellazione in quanto la parte di aggiunta è analoga. La creazione di blocchi attuata dal Translator è implicita.

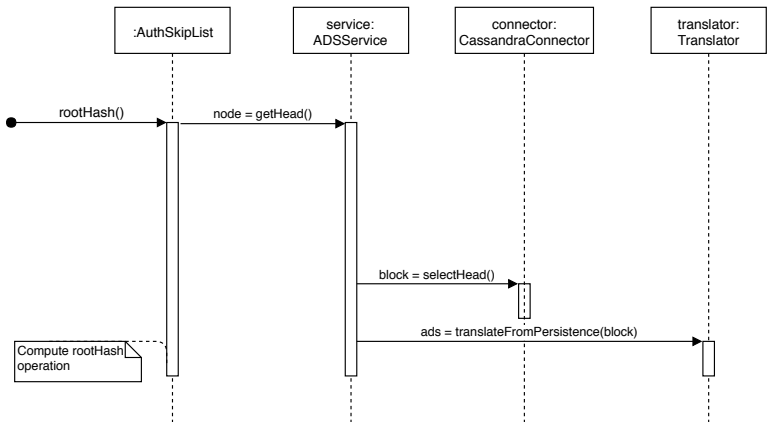


Figura 2.12: Diagramma di sequenza per l'operazione di calcolo del Root Hash.

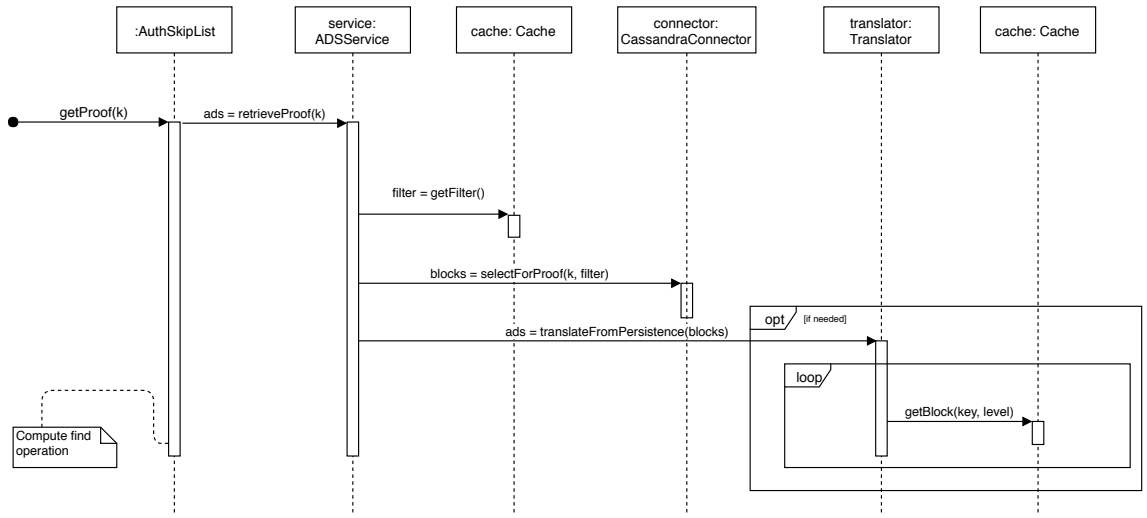


Figura 2.13: Diagramma di sequenza per l'operazione di ottenimento di una Proof valida per una chiave.

Capitolo 3

Realizzazione e testing

3.1 Realizzazione

Come già accennato in fase di progettazione, l'intera libreria è stata realizzata in *Kotlin*, un linguaggio di programmazione basato sulla JVM ed ispirato ad altri linguaggi tra i quali *Scala* e *Java*. Inoltre questa scelta ha permesso una perfetta integrazione con l'ambiente in cui è utilizzata, anch'esso scritto interamente in Kotlin. L'intera progettazione e realizzazione è stata effettuata seguendo sia principi di TDD, ossia *Test-Driven Development*, di cui si approfondirà nel paragrafo successivo, sia di DDD, ovvero *Data-Driven Design*: sono state utilizzate alcune tecniche che hanno permesso di cambiare il comportamento delle API, parametrizzandolo, durante le varie esecuzioni.

3.2 Testing

L'intero sviluppo delle API è stato condotto seguendo principi di TDD. Il testing della libreria è stato condotto in maniera progressiva e incrementale, con lo scopo di verificarne il funzionamento complessivo:

- **Test unitari**: con lo scopo di verificare il funzionamento delle singole unità del sistema.
- **Test di integrazione**: al fine di verificare la comunicazione tra specifiche parti del sistema, in particolare la corretta comunicazione tra la Skip List Autenticata

e la Proof.

- **Test end-to-end**: per verificare il collegamento complessivo tra tutti gli elementi del sistema. Leggera estensione del caso precedente.

- **Test di accettazione**: in questo caso è stato verificato il corretto funzionamento complessivo del sistema, considerandolo dal punto di vista dell'utilizzo. E' stata dunque data una particolare attenzione alle capacità di concorrenza simulando la situazione di accesso concorrente a risorse condivise da parte di attori Akka, con *thread* indipendenti del Sistema Operativo.

Questo ha consentito in più occasioni modifiche, ottimizzazioni e miglioramenti del codice, cosi come flessibilità rispetto alle eventuali richieste di aggiunta di specifiche funzionali.

Il testing inoltre è stato facilitato dalla scrittura di funzioni di stampa dell'intera struttura in maniera grafica su *console*, sia della Skip List Autenticata, sia della Skip List Proof. Questo ha consentito di tracciare facilmente il funzionamento dell'intera libreria anche su dataset molto grandi.

Conclusioni e sviluppi futuri

Nell'ambito di questa attività è stato realizzato un set di API di supporto alla realizzazione di un protocollo per l'integrità dei dati su cloud. La progettazione ha dovuto tenere conto sia dei casi d'uso relativi a requisiti funzionali sia della possibilità di utilizzo dell'intera libreria in contesto concorrente. Oltre a ciò è stato deciso di prendere in considerazione anche altre specifiche come usabilità, prestazioni e sostenibilità, come già ampiamente discusso in fase di analisi. Lo studio teorico e la progettazione sono poi proseguite con lo scopo di realizzare una estensione delle API atta a consentirne la memorizzazione e gestione in persistenza.

Problemi aperti e sviluppi futuri

Limiti concettuali su Proof

Concettualmente parlando, come si è già detto, la Proof rappresenta una struttura a se stante dalla struttura di partenza, a partire dal momento in cui viene creata. Si è inoltre sottolineato come in realtà non ci sia nessuna conoscenza dell'ADS da parte della Proof a posteriori, tanto più considerando il caso in cui l'ADS sia memorizzata in persistenza. Ebbene, nell'ambito dell'utilizzo delle librerie si vuole istanziare una Proof a partire da uno stato iniziale T_0 della struttura. Successivamente alla ricezione e applicazione sull'ADS di blocchi di update, che può aver ricevuto, si vogliono riproporre tali aggiornamenti sulla Proof, parallelamente. Tuttavia l'attuale realizzazione consente questo solo nel caso in cui gli update riguardino aggiornamenti di chiavi già esistenti. Infatti update che aggiungono o cancellano chiavi, modificano la struttura, che, essendo inoltre randomica, può variare consistentemente nella sua interezza. Queste informazioni

non sono più conosciute dalla Proof, dal momento che è una realtà ontologicamente indipendente, anche se sarebbero necessarie. Non è possibile per esempio sapere se il nodo *HEAD* è variato, crescendo o decrescendo, rendendo inconsistente la stessa Proof, anche prima di applicare gli update.

Ipotesi di soluzione con Modello a ruoli

Una ipotesi di soluzione è stata ideata, ma non realizzata. Sarebbe interessante come sviluppo futuro attuarne la realizzazione.

L'idea parte dall'osservazione che nella Skip List Autenticata ogni nodo, in un particolare istante temporale, riveste un ruolo. Questo ruolo inoltre non è statico, ma può cambiare corrispondentemente ad un qualsiasi aggiornamento che comporti una aggiunta o una cancellazione. Dunque la soluzione prevede di eliminare l'assunto di dover creare una struttura a se stante, mantenendo tutte le informazioni necessarie all'interno della struttura principale, al solo costo di dover mantenere aggiornata questa struttura di ruoli. Gli algoritmi delle varie operazioni, di conseguenza, saranno dipendenti dal ruolo dei vari nodi toccati. Così facendo, gli aggiornamenti che arrivano in blocco all'ADS, indirettamente tengono aggiornate tutte le Proof presenti al suo interno, anche e soprattutto nei casi di aggiunta o cancellazione. I ruoli principali di cui tener conto sarebbero:

- **Head**. Rivestito in ogni istante da uno e un solo nodo, ovvero lo *Start Node*. E' il punto di partenza per la generazione di Proof e per tutte le altre operazioni sull'ADS.
- **Tower**. Nodo che ha sopra di se un altro nodo.
- **Plateau**. Nodo che non ha nulla al di sopra di se.
- **Head<Proof>**. Nodo HEAD di riferimento per una particolare Proof, relativa ad un istante temporale precedente ad eventuali variazioni dell'ADS.
- **PathProof<Proof>**. Nodo presente nel path di una determinata Proof.
- **HashingNode<Proof>**. Nodo non presente nel path, ma necessario per ricostruzione del RootHash per una determinata Proof.

Realizzazione progetto persistenza

Infine, un ulteriore sviluppo futuro, potrebbe vedere la realizzazione e l'applicazione ad un caso reale del progetto di persistenza presentato nel corso di questa tesi. Infatti, sebbene questo sia stato condotto con particolare riferimento a *Cassandra*, un DBMS non relazionale molto diffuso, in realtà con poche dovute accortezze è perfettamente replicabile per molti altre situazioni, risultando quasi totalmente indipendente dalla tecnologia adottata.

Bibliografia

[BFM14] Michael Kaminsky Bin Fan, David G. Andersen and Michael D. Mitzenmacher. Harvard university. Technical report, 2014.

[FGP18] Diego Pennino Federico Griscioli and Maurizio Pizzonia. *On-the-Fly Integrity Checks by Authenticated Data Structures without impairing scalability.* 2018.

[FH07] Keir Fraser and Tim Harris. *Concurrent Programming Without Locks.* Technical report, 2007.

[GT01] Michael T. Goodrich and Roberto Tamassia. *Efficient Authenticated Dictionaries with Skip Lists and Commutative Hashing.* Technical report, 2001.

[Mon18] Gianmaria Del Monte. Studio e realizzazione di un protocollo efficiente per l'integrità dei dati su cloud, 2018.

[Piz17] Maurizio Pizzonia. Authenticated data structures. Lucidi del corso *Sicurezza dei sistemi informatici e delle reti*, 2017.